

Improved Cryptanalysis of the Permuted Kernel Problem with Applications to PERK v2.2.0, SUSHSYFISH and PKP-DSS

Abul Kalam¹, Sudeshna Karmakar¹, Arindam Mukherjee², Soumya Sahoo¹, and Santanu Sarkar¹

¹ Indian Institute of Technology Madras, India

² Achhruram Memorial College, Jhalda, India

abulkalam.sunny@gmail.com, sudeshnakarmakar.dgp@gmail.com,
ma18d004@smail.iitm.ac.in, soumyasahoo078@gmail.com,
sarkar.santanu.birl@gmail.com

Abstract. The Permuted Kernel Problem (PKP), introduced by Shamir, is a computationally hard problem underlying recent post-quantum signature schemes such as PERK, SUSHSYFISH and PKP-DSS. Among these, PERK is a second-round candidate in NIST’s Additional Digital Signature Schemes post-quantum standardization process, SUSHSYFISH appeared at EUROCRYPT 2020, and PKP-DSS was one of the finalists in the CACR competition. The best known attacks on PKP rely on combinatorial meet-in-the-middle techniques, notably the algorithms of Koussa, Macario-Rat, and Patarin (KMP) and Santini, Baldi, and Chiaraluce (SBC), whose complexities remain super-exponential, with memory requirements of the same order as their time complexities.

In this work, we obtain improved cryptanalytic results for PKP that strictly outperform all previously known attacks. In particular, although no parameter sets of PERK v2.2.0 fall below the NIST security levels, we provide the first evidence that secret vector recovery for all PERK v2.2.0 parameter sets can be achieved with complexity below their estimated bit-security levels. We additionally obtain improved bit-complexity estimates for the SUSHSYFISH and PKP-DSS parameter sets. We further introduce a Schroeppe–Shamir style time–memory trade-off in the PKP setting. Although PKP does not admit square-root memory as in the classical subset-sum problem, our adaptation achieves substantial memory reductions while keeping the time complexity close to the best known attacks. Overall, our results provide improved cryptanalytic insight into PKP and refine the current understanding of the concrete security of PKP-based signature schemes.

Keywords: PKP · meet-in-the-middle · time-memory trade-off.

1 Introduction

The Permuted Kernel Problem (PKP) is a computationally hard problem introduced by Shamir in 1989 [29]. Given a matrix $\mathbf{A} \in \mathbb{F}_q^{n \times n}$ and a vector $\mathbf{b} \in \mathbb{F}_q^n$,

the basic form of PKP asks for a permutation π such that the permuted vector $\pi(\mathbf{b})$ lies in the kernel of $\mathbf{A}$, that is, $\mathbf{A}\pi(\mathbf{b}) = \mathbf{0}$. This combination of structural simplicity and resistance to efficient algorithms makes PKP both practical to implement and theoretically appealing. Despite its simple formulation, PKP is known to be NP-complete [16]. Heuristically, this suggests that PKP remains hard even for quantum computers, since Grover’s algorithm [15] provides at most a quadratic speedup for NP-complete problems.

Over the years, several variants of PKP have been proposed. These include the inhomogeneous PKP, where the right-hand side of the equation is a nonzero vector, and the multi-dimensional PKP [21], in which the vector $\mathbf{b}$ is replaced by a matrix. More recently, Bettaieb *et al.* introduced a relaxed version of PKP (r-IPKP) [5]. Importantly, these generalized formulations also remain NP-complete [4].

As most standardized post-quantum cryptographic schemes rely on hardness assumptions derived from structured lattices, the independence of PKP from the lattice framework provides a valuable alternative. This diversification is important in the post-quantum standardization effort, since it reduces reliance on a single mathematical foundation. As a result, PKP has emerged as a strong and versatile hardness assumption for constructing post-quantum secure digital signatures.

PKP-Based Signature Schemes. Several signature schemes now base their security on random instances of PKP. PKP-DSS [9] is a notable example and was selected as one of the finalists in the competition organized by the Chinese Association for Cryptographic Research (CACR). The SUSHSYFISH signature scheme [8] also relies on the hardness of PKP and illustrates the flexibility of the assumption. Another example is the construction by Bidoux and Gaborit [10], which adopts PKP as its core security foundation. The PERK [5] family of signature schemes has played a significant role in the development of PKP-based cryptography. The initial version of PERK relied on a relaxed variant of the problem. The most recent release, PERK v2.2.0 [1], announced in December 2025, is instead based on the standard formulation of PKP. The scheme is currently in the second-round of the NIST Post-Quantum Cryptography process, reflecting its practical relevance and maturity. PKP has also been used in signature constructions based on the MPC-in-the-Head paradigm [6]. Moreover, the growing interest in PKP-based cryptographic schemes has led to the inclusion of PKP in the CryptographicEstimators [13] maintained by TII, reflecting its recognition as a relevant post-quantum hardness assumption. Related notions, such as Permutation Equivalence of Codes and Kernel (PECK), have also been introduced and employed, for example in the SPECK signature scheme [2].

In most PKP-based schemes, a random instance of the Permuted Kernel Problem serves as the public key, while the corresponding solution acts as the secret key. The parameters $\tilde{n}$, n , and q are typically chosen so that a random instance has, on average, a unique solution. As a general heuristic, instances with more solutions tend to be easier to attack. The distribution of the number of solutions for random PKP instances has recently been analyzed in depth by Sanna [25].

State-of-the-Art in PKP Cryptanalysis. Over the years, several cryptanalytic approaches have been developed to weaken the computational hardness of PKP. These include purely linear algebraic methods, combinatorial key-search strategies based on meet-in-the-middle techniques, and hybrid approaches that integrate both ideas. The first non-trivial attack after exhaustive search was proposed by Georgiades [14], who reduced the matrix and then guessed the free variables in the resulting system. A time-memory trade-off was later introduced by Baritaud, Campana, Chauvaud, and Gilbert (BCCG) [3]. Patarin and Chauvaud subsequently combined the ideas of Georgiades and BCCG to obtain a more efficient attack [24]. A notable step forward came with the time-memory trade-off proposed by Jaulmes and Joux in [17], which was, for some time, regarded as the most efficient attack on PKP. However, it was later shown that their complexity estimates were overly optimistic [20].

A substantive advancement occurred in 2019 with the work of Koussa, Macario-Rat, and Patarin (KMP) [20]. Their key idea is to decompose the problem into smaller instances, apply a meet-in-the-middle strategy to recover a partial solution, and then use structural relations among the reduced instances to reconstruct the full solution. Further progress was made by Santini, Baldi, and Chiaraluce (SBC) [26], who refined KMP by incorporating an Information Set Decoding (ISD) style preprocessing step, similar to the approach in [7], to avoid exhaustively generating the second list in the KMP algorithm. The improvements over KMP in the mono-dimensional case are modest for small values of n , and for larger values of n the SBC algorithm can perform worse than the KMP algorithm. Santini *et al.* later extended this method [27] to the multi-dimensional variant of PKP. More recently, an improved cryptanalysis of PKP was proposed by Budroni *et al.* [11]. Additionally, the binary variant of PKP has been investigated by Paiva *et al.* in [23].

Despite these advances, all known attacks against PKP remain super-exponential in complexity. As noted in the latest version (v2.2.0) of PERK [1],

“... Furthermore the KMP algorithm suffers from a memory complexity that is equal to its time complexity. Therefore, realistic attacks have to resort to time-memory trade-offs further adding to this margin.”

Motivated by this observation, in this work we design a time-memory trade-off for PKP inspired by the classical Schroeppel-Shamir framework [28].

Cost Model. For the purpose of analyzing algorithmic complexity, we adopt the standard cost model in which multiplying an $(m \times n)$ matrix by an $(n \times 1)$ vector incurs a cost of $\mathcal{O}(mn)$ [22]. Moreover, intersecting two sets of sizes ℓ and r is modeled as taking $\mathcal{O}(\min\{\ell, r\})$ time [12]. Finally, all $\mathbb{F}_q$ operation counts are converted into bit-complexity by multiplying the resulting costs by a factor of $\log_2(q)$ [13].

1.1 Our Contribution

For the sake of clarity and completeness, we begin by recalling the KMP [20] and SBC [26] algorithms, which serve both as foundational building blocks for our

methods and as baseline points of comparison for our algorithmic improvements. As an initial contribution, we provide an accessible and intuitive reinterpretation of the KMP and SBC algorithms within the framework of combinatorial meet-in-the-middle attack strategies, together with a clear and concrete analysis of their complexities.

Secret Vector Recovery for PKP. Our main technical contribution is a new algorithm for solving the PKP problem. Given a matrix $\mathbf{A} \in \mathbb{F}_q^{\tilde{n} \times n}$ and a vector $\mathbf{b} \in \mathbb{F}_q^n$, the approach follows the general structure of the KMP algorithm by first transforming the system into a semi-systematic form and reducing the problem to a smaller subsystem. The key novelty arises in solving this reduced instance.

Whereas KMP constructs two lists $\mathcal{L}_1$ and $\mathcal{L}_2$ using naive matrix–vector multiplication and performs filtering only after a meet-in-the-middle merge, our method introduces a constraint-based enumeration strategy that restricts the candidate vectors in both lists. The lists are generated efficiently via incremental updates based on Steinhaus–Johnson–Trotter (SJT) [18] enumeration, and a structured compatibility table derived from ranks of index-support sets is used to significantly accelerate the filtering phase. We further study an analogous adaptation of this strategy to the SBC algorithm.

Using this framework, we analyze the resulting time bit-complexity exponents for the signature schemes PERK v2.2.0 [1], SUSHSYFISH [8] and PKP-DSS [9] demonstrating that our algorithms strictly outperform both KMP and SBC. A comparison of time and memory bit-complexity exponents is summarized in Table 1. In particular, the results indicate that

- the secret vector of the NIST Round 2 candidate PERK can be recovered below the estimated bit-security levels([1], Section 7.2.5) for all proposed parameter sets; more precisely, the 150-bit, 220-bit, and 286-bit estimated security levels can be achieved in time $2^{145.94}$, $2^{216.49}$, and $2^{281.32}$, respectively;
- the bit-complexities for the SUSHSYFISH and PKP-DSS parameter sets are likewise reduced compared to the best previously known attacks.

At the same time, we emphasize that none of the PERK v2.2.0 parameter sets fall below the claimed NIST security levels. As a further contribution, we show that allowing only a small increase in the time complexity relative to KMP yields memory reductions of nearly **18%** relative to the memory exponents of KMP across all parameter sets of the considered signature schemes. These reductions constitute a practically meaningful improvement.

Time–Memory Trade-off for PKP. A standard approach for significantly reducing memory in combinatorial attacks is to employ the Schroeppe–Shamir technique. In this work, we present an adaptation of a Schroeppe–Shamir style meet-in-the-middle time–memory trade-off to the PKP setting. It is important to note that our adaptation is not a direct transcription of the classical subset-sum version. In contrast to the subset-sum setting, where the Schroeppe–Shamir framework achieves square-root memory, the structure of PKP inherently requires higher memory.

From a technical perspective, following the Schroeppe–Shamir paradigm, we employ a four-way merge to recover solutions of the reduced system. All

Table 1. Summary of time and memory bit-complexity exponents for existing algorithms and this work, applied to the PERK v2.2.0, SUSHSYFISH and PKP-DSS parameter sets.

Schemes	(n, m, q)	Existing works [20, 26]		This work (Algorithm 1 and 3)	
		Time	Memory	Time	Memory
PERK v2.2.0	64, 28, 2048	150.60	144.01	145.94	143.08
	92, 44, 2048	222.32	213.55	216.49	210.43
	118, 60, 2048	286.88	278.91	281.32	277.24
SUSHSYFISH	61, 29, 997	137.64	131.25	133.18	130.28
	87, 43, 1409	204.66	198.56	199.69	190.02
	111, 56, 1889	268.60	262.04	263.31	262.50
PKP-DSS	69, 42, 251	134.78	124.69	131.16	122.44
	94, 55, 509	199.24	190.68	196.31	184.51
	106, 48, 4093	270.21	258.83	262.75	253.00

intermediate lists are generated under suitable constraints, and their elements are enumerated efficiently using the SJT algorithm. When applied to the PERK v2.2.0, SUSHSYFISH and PKP-DSS parameter sets, this adaptation yields substantial memory reductions while keeping the time complexity close to those of the best previously known attacks. These results demonstrate that our approach provides a meaningful and previously unexplored method for lowering the memory complexity of attacks on PKP.

Finally, we validate our claims³ by a proof-of-concept implementation. We tested our algorithms on 1000 random PKP instances and successfully recovered the secret in every case.

The scripts used for the experimental verification of Algorithm 1 and Algorithm 2 are available on <https://github.com/Abulkalam-sunny/Improved-Cryptanalysis-of-the-Permuted-Kernel-Problem.git>

Paper Layout. The paper is organized as follows. Section 2 recalls the SJT algorithm and introduces the PKP problem. Section 3 reviews existing attacks. Section 4 presents our algorithm for solving PKP. Section 5 develops a time-memory trade-off based on a Schroeppe-Shamir style adaptation. Section 6 provides numerical results for selected cryptographic parameter sets. Finally, Section 7 concludes the paper.

2 Preliminaries

Here we introduce the notation used throughout the paper. Bold capital letters (e.g., $\mathbf{M}$) denote matrices, and bold lowercase letters (e.g., $\mathbf{v}$) denote vectors.

³Our complexity analysis relies only on the standard heuristic assumptions commonly used in meet-in-the-middle attacks.

The number of ordered selections of m elements from a set of size n (without replacement) is written as ${}^nP_m = \frac{n!}{(n-m)!}$. The finite field with q elements, where q is a prime power, is denoted by $\mathbb{F}_q$. We denote the set $\{0, 1, \dots, n-1\}$ by $[n]$. For a vector $\mathbf{v} \in \mathbb{F}_q^n$ with distinct entries, we define $\mathcal{S}_\ell(\mathbf{v})$ as the set of all length- ℓ vectors formed by selecting ℓ entries of $\mathbf{v}$ without replacement; its cardinality is $|\mathcal{S}_\ell(\mathbf{v})| = {}^nP_\ell$. For a matrix $\mathbf{M}$, the notation $\mathbf{M}[:n]$ refers to the submatrix consisting of its first n columns, and $\mathbf{M}[n:]$ refers to the submatrix from the $(n+1)^{\text{th}}$ column onward. For a vector $\mathbf{v}$, $\mathbf{v}[:n]$ denotes the subvector of its first n components, and $\mathbf{v}[n:]$ denotes the subvector from the $(n+1)^{\text{th}}$ component to the end. We use the support of a vector to denote the set of its entries. For a vector $\mathbf{v} = (v_0, v_1, \dots, v_{n-1})$, we write $\text{Supp}(\mathbf{v}) = \{v_0, v_1, \dots, v_{n-1}\}$. Finally, $\mathcal{S}_n$ denotes the set of all permutations of n elements.

2.1 Steinhaus–Johnson–Trotter Algorithm

The Steinhaus–Johnson–Trotter (SJT) algorithm [18] is a classical and well-studied procedure for generating all permutations of a finite set in a highly structured manner. Its defining characteristic is that each successive permutation differs from the previous one by exactly one adjacent transposition. This minimal-change property makes the algorithm particularly attractive in combinatorial enumeration and algorithmic settings where local updates are preferred.

Given a set of n distinct elements, the SJT algorithm enumerates all $n!$ permutations by repeatedly swapping adjacent elements. Each element is assigned a direction, either left or right. At every step, the algorithm identifies the *largest mobile element*, defined as the largest element that can move in its assigned direction without leaving the array or violating the relative ordering of larger elements. Once identified, this element is swapped with its adjacent neighbor in the indicated direction. Subsequently, the directions of all elements larger than the moved element are reversed.

More precisely, while a mobile element exists, the algorithm iterates through the following steps:

1. Identify the largest mobile element k .
2. Swap k with the adjacent element in the direction it is facing.
3. Reverse the directions of all elements larger than k .

The process terminates when no mobile elements remain, at which point all permutations have been generated. The algorithm requires linear space and runs in $\mathcal{O}(n \cdot n!)$ time, as each of the $n!$ permutations is produced with an $\mathcal{O}(n)$ update step.

2.2 Permuted Kernel Problem

The **Permuted Kernel Problem (PKP)** is a computational problem in linear algebra and combinatorics, first introduced [29] by Adi Shamir in 1989. It is formally defined as follows:

Definition 1. Given a random matrix $\mathbf{A} \in \mathbb{F}_q^{\tilde{n} \times n}$ and a random vector $\mathbf{b} \in \mathbb{F}_q^n$, the PKP asks for a permutation $\pi \in \mathcal{S}_n$ such that the permuted vector $\pi(\mathbf{b})$ lies in the kernel (null space) of $\mathbf{A}$, i.e.,

$$\mathbf{A}\pi(\mathbf{b}) = \mathbf{0}.$$

This formulation is commonly referred to as the *mono-dimensional homogeneous* PKP, since it involves a single input vector and a zero target vector. Despite its seemingly simple structure, this version of the problem exhibits considerable combinatorial complexity and has been extensively studied in the context of code-based and permutation-based cryptographic schemes.

A natural extension of this problem is the *multi-dimensional* PKP, in which the input is a matrix (rather than a single vector $\mathbf{b}$), and one seeks a column-wise permutation that maps multiple vectors into the kernel of $\mathbf{A}$. Both the mono- and multi-dimensional variants are often called *homogeneous*, as they target the zero vector (or zero matrix) on the right-hand side. A further generalization yields the *inhomogeneous* PKP, where the right-hand side is a nonzero vector or matrix. These generalizations broaden the applicability of the PKP to more complex and potentially more secure cryptographic constructions, while preserving the underlying hardness that makes the problem attractive for post-quantum security.

A majority of PKP-based schemes rely on the mono-dimensional PKP variant; we therefore restrict our attention to this setting. Additionally, we assume that all entries of the input vector $\mathbf{b}$ are distinct and the given instance has exactly one solution. These assumptions maximize the permutation space, thus representing the most challenging instance of the problem.

Throughout the remainder of this work, we consider instances of the PKP problem defined by a matrix $\mathbf{A} \in \mathbb{F}_q^{\tilde{n} \times n}$ and a vector $\mathbf{b} \in \mathbb{F}_q^n$, where

$$\mathbf{b} = (b_0, b_1, \dots, b_{n-1})^T.$$

We denote by $\mathbf{x}^\star = \pi(\mathbf{b})$ a target solution of the instance, and use the superscript $\star$ to indicate a solution, or a partial solution, throughout the paper. We write $\mathbf{0}$ for the zero vector or zero matrix over $\mathbb{F}_q$, depending on context, and define $\mathbf{1} = (1, 1, \dots, 1)^T \in \mathbb{F}_q^n$ as the all-one vector.

3 The Existing Attacks on PKP

In this section, we review the established algorithmic strategies for solving PKP, with particular emphasis on the recent state-of-the-art analyses by Koussa *et al.* and Santini *et al.*, which currently provide the most efficient known attacks on PKP instances.

3.1 The KMP Algorithm

A notable approach for solving the PKP was proposed by Koussa, Macario-Rat, and Patarin [20]. Their approach leverages the fact that permutations preserve

the sum of a vector's components; hence, the sum σ of $\pi(\mathbf{b})$ is known and fixed. This provides an additional linear equation, and the system can be rewritten as:

$$\tilde{\mathbf{A}}\mathbf{x} = (\sigma, 0, \dots, 0)^T, \quad (1)$$

where, $\mathbf{x}^* = \pi(\mathbf{b})$ is the desired solution of the above equation, the augmented matrix $\tilde{\mathbf{A}} \in \mathbb{F}_q^{m \times n}$ is defined as:

$$\tilde{\mathbf{A}} = \begin{pmatrix} \mathbf{1}^T \\ \mathbf{A} \end{pmatrix},$$

and, $m = \tilde{n} + 1$. The algorithm begins by transforming $\tilde{\mathbf{A}}$ into a semi-systematic form via multiplication by an invertible matrix $\mathbf{Q} \in \mathbb{F}_q^{m \times m}$:

$$\mathbf{Q}\tilde{\mathbf{A}} = \begin{pmatrix} \mathbf{I}_{m-k} & \mathbf{M}' \\ \mathbf{0} & \mathbf{M} \end{pmatrix}, \quad (2)$$

where $\mathbf{M}' \in \mathbb{F}_q^{m-k \times w}$, $\mathbf{M} \in \mathbb{F}_q^{k \times w}$, $w = n - m + k$ and k is an optimization parameter. The right-hand side of Eq. (1) transforms accordingly:

$$\mathbf{t} = \mathbf{Q}(\sigma, 0, \dots, 0)^T.$$

The vector $\mathbf{x}^* \in \mathbb{F}_q^n$ is then split into two parts: $\mathbf{x}^* = (\mathbf{x}_1^* \parallel \mathbf{x}_2^*)$ with $\mathbf{x}_1^* \in \mathbb{F}_q^{m-k}$ and $\mathbf{x}_2^* \in \mathbb{F}_q^w$. Substituting into the system gives two equations:

$$\mathbf{x}_1 = \mathbf{t}_1 - \mathbf{M}'\mathbf{x}_2, \quad (3)$$

$$\mathbf{M}\mathbf{x}_2 = \mathbf{t}_2, \quad (4)$$

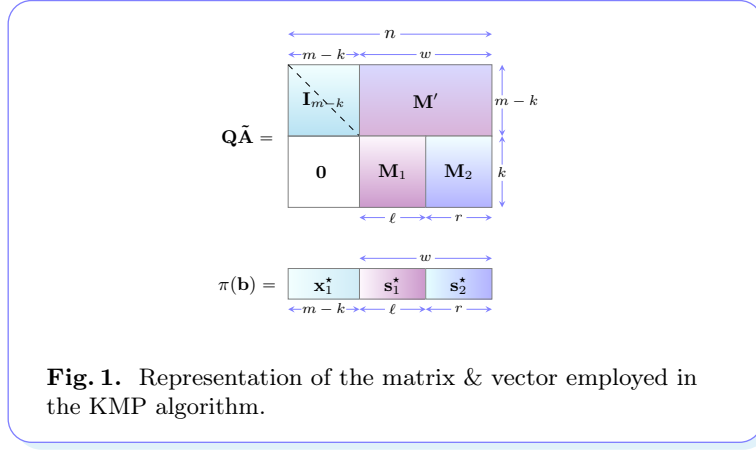
where $\mathbf{t} = (\mathbf{t}_1 \parallel \mathbf{t}_2)$ with $\mathbf{t}_1 \in \mathbb{F}_q^{m-k}$, $\mathbf{t}_2 \in \mathbb{F}_q^k$. To obtain the desired solution, for each vector $\mathbf{x}_2 \in \mathcal{S}_w(\mathbf{b})$ satisfying Eq. (4), we compute the corresponding $\mathbf{x}_1$ using Eq. (3), and then verify whether the concatenated vector $(\mathbf{x}_1 \parallel \mathbf{x}_2)$ forms a permutation of $\mathbf{b}$. Therefore, the main computational challenge lies in solving Eq. (4).

To efficiently retrieve the $\mathbf{x}_2^*$ vector, the KMP algorithm employs a meet-in-the-middle strategy. The vector $\mathbf{x}_2^*$ is split as $\mathbf{x}_2^* = (\mathbf{s}_1^* \parallel \mathbf{s}_2^*)$, with $\mathbf{s}_1^* \in \mathbb{F}_q^\ell$ and $\mathbf{s}_2^* \in \mathbb{F}_q^r$, where ℓ is an optimization parameter such that $\ell + r = w$. The matrix $\mathbf{M}$ is accordingly partitioned as $\mathbf{M} = (\mathbf{M}_1 \parallel \mathbf{M}_2)$, allowing Eq. (4) to be rewritten as:

$$\mathbf{M}_1\mathbf{s}_1 = \mathbf{t}_2 - \mathbf{M}_2\mathbf{s}_2. \quad (5)$$

The structure of the matrices and vectors used in the algorithm is illustrated in Figure 1.

The algorithm proceeds by enumerating all candidate vectors $\mathbf{s}_1 \in \mathcal{S}_\ell(\mathbf{b})$ and computing the left-hand side of Eq. (5), storing each result in a list $\mathcal{L}$. Then, for each candidate vector $\mathbf{s}_2 \in \mathcal{S}_r(\mathbf{b})$, it computes the right-hand side $\mathbf{t}_2 - \mathbf{M}_2\mathbf{s}_2$ and checks for a match in $\mathcal{L}$. When a match is found, a candidate $\mathbf{x}_2 = (\mathbf{s}_1 \parallel \mathbf{s}_2)$



is constructed. If this $\mathbf{x}_2$ is a valid w -permutation of $\mathbf{b}$, i.e., $\mathbf{x}_2 \in \mathcal{S}_w(\mathbf{b})$, the corresponding $\mathbf{x}_1$ is computed via Eq. (3). Finally, if $(\mathbf{x}_1 \parallel \mathbf{x}_2) \in \mathcal{S}_n(\mathbf{b})$, i.e., $\mathbf{x}_1$ and $\mathbf{x}_2$ are *compatible*, the algorithm returns it as the solution to the PKP instance.

Remark 1. To determine whether two vectors of lengths ℓ and r are *compatible*, we compute the intersection of their supports and verify that it is empty. This operation requires $\mathcal{O}(\min\{\ell, r\})$ time [12].

Complexity analysis of KMP Algorithm. To generate the list $\mathcal{L}$, the algorithm enumerates all vectors $\mathbf{s}_1 \in \mathcal{S}_\ell(\mathbf{b})$ and computes $\mathbf{M}_1 \mathbf{s}_1$. Since $|\mathcal{S}_\ell(\mathbf{b})| = {}^n P_\ell$ and each matrix multiplication requires a cost of $k\ell$, the total cost of generating $\mathcal{L}$ is

$$\mathcal{O}\left(k\ell \cdot {}^n P_\ell\right).$$

Next, the algorithm enumerates all vectors $\mathbf{s}_2 \in \mathcal{S}_r(\mathbf{b})$ to compute $\mathbf{t}_2 - \mathbf{M}_2 \mathbf{s}_2$ and check for collisions in $\mathcal{L}$. As $|\mathcal{S}_r(\mathbf{b})| = {}^n P_r$, this step requires cost

$$\mathcal{O}\left(kr \cdot {}^n P_r\right).$$

For each collision found, the algorithm performs a compatibility check to combine $\mathbf{s}_1$ and $\mathbf{s}_2$ into a valid solution of Eq. (4). The expected number of collisions is $\frac{{}^n P_\ell {}^n P_r}{q^k}$, and since a compatibility test costs $\min\{\ell, r\}$, this step requires an expected cost of

$$\mathcal{O}\left(\min\{\ell, r\} \cdot \frac{{}^n P_\ell {}^n P_r}{q^k}\right).$$

After obtaining valid candidates $\mathbf{x}_2$, the algorithm computes $\mathbf{x}_1 = \mathbf{t}_1 - \mathbf{M}' \mathbf{x}_2$ and checks the compatibility between $\mathbf{x}_1$ and $\mathbf{x}_2$. Since the expected number of

valid $\mathbf{x}_2$ is $\frac{{}^nP_{\ell+r}}{q^k}$ and the cost of each such computation is $(m-k)w + \min\{m-k, w\}$, the expected cost of this step is

$$\mathcal{O}\left(\left((m-k)w + \min\{m-k, w\}\right) \cdot \frac{{}^nP_{\ell+r}}{q^k}\right).$$

Combining all contributions, the overall time complexity of the KMP algorithm is

$$T_{\text{KMP}} = \mathcal{O}\left(k\ell \cdot {}^nP_\ell + kr \cdot {}^nP_r + \min\{\ell, r\} \cdot \frac{{}^nP_\ell {}^nP_r}{q^k} + \left((m-k)w + \min\{m-k, w\}\right) \cdot \frac{{}^nP_{\ell+r}}{q^k}\right).$$

Finally, the memory usage is dominated by the storage of the list $\mathcal{L}$, which contains ${}^nP_\ell$ entries. It is worth noting that we do not store the pair $(\mathbf{M}_1\mathbf{s}_1, \mathbf{s}_1)$ explicitly in $\mathcal{L}$. Instead, we retain only $\mathbf{s}_1$ and store a hash of $\mathbf{M}_1\mathbf{s}_1$ as the lookup key. Since the cost of hashing is negligible compared to a field operation, it does not significantly affect the overall complexity. Consequently, the memory requirement of the KMP algorithm reduces to storing only ${}^nP_\ell$ many ℓ -vectors.

3.2 The SBC Algorithm

An algorithmic improvement of the KMP method was recently proposed by Santini, Baldi, and Chiaraluce in [26], introducing a preprocessing step to enhance performance. This approach was further extended in [27] to address the multi-dimensional PKP.

As previously discussed, the KMP algorithm determines d linear equations involving $n-m+d$ variables, where d is a parameter of the algorithm.. In contrast, the method introduced by Santini *et al.* seeks to construct d equations whose support size is restricted to $w \leq n-m+d$ variables, thereby potentially reducing the number of variables involved. This objective is achieved through adaptations of Information Set Decoding (ISD) techniques [7]. When $w < n-m+d$, the reduced support leads to a smaller search space, significantly lowering the enumeration effort required to identify valid assignments to the variables involved. However, if the support size reaches $w = n-m+d$, the SBC approach becomes equivalent to the original KMP algorithm, yielding no improvement in computational complexity. The SBC algorithm is outlined in Appendix G.

4 A New Approach for Solving PKP

In this section, we present a new algorithm for solving the PKP problem. The key idea is to shift a portion of the computation to a preprocessing phase. Rather than computing $\mathbf{M}_1\mathbf{s}_1$ and $\mathbf{t}_2 - \mathbf{M}_2\mathbf{s}_2$ for all $\mathbf{s}_1 \in \mathcal{S}_\ell(\mathbf{b})$ and $\mathbf{s}_2 \in \mathcal{S}_r(\mathbf{b})$, respectively, using naive matrix-vector multiplication, and performing compatibility checks only after completing the meet-in-the-middle search as in KMP, we adopt a preprocessing-based approach.

By exploiting the SJT algorithm, the quantities $\mathbf{M}_1\mathbf{s}_1$ and $\mathbf{t}_2 - \mathbf{M}_2\mathbf{s}_2$ are computed incrementally rather than from scratch. In addition, a two-dimensional

array is constructed in advance to record the compatibility of each pair consisting of an ℓ -vector and an r -vector. These steps enable efficient retrieval of compatible candidate pairs during the search phase. We describe them in detail before presenting the algorithm.

4.1 Incremental Enumeration via the SJT Algorithm

Consider a fixed matrix $\mathbf{H} \in \mathbb{F}_q^{u \times v}$ and a fixed vector $\mathbf{v} \in \mathbb{F}_q^v$, and suppose we wish to compute the set $\{\mathbf{H}\mathbf{x} \mid \mathbf{x} \in \mathcal{S}_v(\mathbf{v})\}$. A naive approach computes $\mathbf{H}\mathbf{x}$ independently for each $\mathbf{x} \in \mathcal{S}_v(\mathbf{v})$, incurring a cost of $\mathcal{O}(uv)$ per matrix–vector multiplication and thus a total cost of $\mathcal{O}(uv \cdot v!)$.

Instead, we enumerate the vectors $\mathbf{x} \in \mathcal{S}_v(\mathbf{v})$ by repeatedly swapping adjacent elements, which produces an ordered sequence $\{\mathbf{x}_0, \mathbf{x}_1, \dots\}$ of permutations. By construction, any two consecutive vectors $\mathbf{x}_i$ and $\mathbf{x}_{i+1}$ differ by exactly one adjacent transposition. More precisely, there exists an index t such that

$$x_{i,j} = x_{i+1,j} \quad \text{for all } j \notin \{t, t+1\}, \quad \text{and} \quad (x_{i,t}, x_{i,t+1}) = (x_{i+1,t+1}, x_{i+1,t}).$$

To generate the sequence $\{\mathbf{x}_i\}$ efficiently, we precompute the *swap_sequence*, defined as the sequence of swap positions produced by applying the SJT algorithm to $(1, 2, \dots, v)$.⁴ The sequence $\{\mathbf{x}_i\}$ can then be generated using this *swap_sequence* according to the rule

$$\mathbf{x}_0 = \mathbf{v} \quad \text{and} \quad \mathbf{x}_{i+1} = \text{Swap}(\mathbf{x}_i, \text{swap_sequence}[i]),$$

where, $\text{Swap}(\mathbf{x}_i, t)$ denotes swapping the entries of $\mathbf{x}_i$ at positions t and $t+1$.

We define $\mathbf{w}_i$ as the product $\mathbf{H}\mathbf{x}_i$. Then $\mathbf{w}_{i+1}$ can be computed from $\mathbf{w}_i$ via an incremental update:

$$\mathbf{w}_{i+1} = \mathbf{w}_i + (x_{i,t+1} - x_{i,t})(\mathbf{H}_t - \mathbf{H}_{t+1}),$$

where, $\mathbf{H}_j$ denotes the j -th column of $\mathbf{H}$. To accelerate these updates, we precompute $(x_1 - x_2)(\mathbf{H}_t - \mathbf{H}_{t+1})$ for all $x_1, x_2 \in \text{Supp}(\mathbf{v})$ and all $t \in [v]$. This precomputation requires $\mathcal{O}(v^3u)$ time and space. After an initial matrix–vector multiplication $\mathbf{w}_0 = \mathbf{H}\mathbf{x}_0$, each subsequent update can then be performed using only $\mathcal{O}(u)$ field operations.

Consequently, the total time complexity of the precomputation phase is $\mathcal{O}(v \cdot v! + v^3u)$, where the first term corresponds to generating the sequence *swap_sequence* of swap positions produced by the SJT algorithm, and the second term accounts for the vector precomputations. The time required to compute the entire set $\{\mathbf{H}\mathbf{x} \mid \mathbf{x} \in \mathcal{S}_v(\mathbf{v})\}$ is $\mathcal{O}(uv + u \cdot v!)$, where the first term arises from the initial matrix–vector multiplication and the second from the incremental updates.

For sufficiently large v , the factorial growth dominates. As a result, the complexities become $\mathcal{O}(v \cdot v!)$ for the precomputation phase and $\mathcal{O}(u \cdot v!)$ for generating the complete list of products. This represents a substantial improvement over the naive $\mathcal{O}(uv \cdot v!)$ approach.

⁴When the SJT algorithm is applied to $(1, 2, \dots, v)$, each step swaps two adjacent positions $(t, t+1)$; the *swap_sequence* records the value t corresponding to each swap.

4.2 Precomputation of the Compatibility Table

The construction of the Compatibility Table for vector pairs in $\mathcal{S}_\ell(\mathbf{b}) \times \mathcal{S}_r(\mathbf{b})$ begins with the notion of index-support (I-Supp) of a vector. For a vector $\mathbf{s}_1 \in \mathcal{S}_\ell(\mathbf{b})$, we define

$$\text{I-Supp}(\mathbf{s}_1) = \{i \in [n] \mid b_i \in \text{Supp}(\mathbf{s}_1)\}.$$

Using this notation, two vectors $\mathbf{s}_1 \in \mathcal{S}_\ell(\mathbf{b})$ and $\mathbf{s}_2 \in \mathcal{S}_r(\mathbf{b})$ are *compatible* if and only if

$$\text{I-Supp}(\mathbf{s}_1) \cap \text{I-Supp}(\mathbf{s}_2) = \emptyset.$$

This compatibility condition forms the basis for constructing the Compatibility Table. An illustrative example of I-Supp is provided in Appendix A.

To organize index-support sets of fixed cardinality in a systematic manner, we introduce a lexicographic ordering on subsets of $[n]$ of size $t \in \{\ell, r\}$. Let $S = \{p_0, p_1, \dots, p_{t-1}\}$ and $S' = \{p'_0, p'_1, \dots, p'_{t-1}\}$ be two t -subsets of $[n]$, each listed in increasing order. We define

$$S < S' \iff \exists k \text{ with } 0 \leq k < t \text{ such that } p_i = p'_i \text{ for all } i < k \text{ and } p_k < p'_k.$$

This ordering is the standard lexicographic order on t -subsets and has been extensively studied in the literature [19]. Once this ordering is established, every t -subset of $[n]$ can be assigned a unique position in the ordered list of all t -subsets. To formalize this, we define the rank function

$$\begin{aligned} \rho_t : \{S \subseteq [n] \mid |S| = t\} &\longrightarrow \left[\binom{n}{t} \right] \quad \text{as,} \\ S = \{p_0, p_1, \dots, p_{t-1}\} &\longmapsto \sum_{j=0}^{t-1} \left[\binom{n - p_{j-1} - 1}{t - j} - \binom{n - p_j}{t - j} \right], \end{aligned}$$

where $p_{-1} = -1$ and $0 \leq p_0 < p_1 < \dots < p_{t-1} \leq n-1$. The value $\rho_t(S)$ is exactly the rank of S in the ordered list of all t -subsets of $[n]$. A proof of correctness of the rank function is provided in Appendix B.

Using this ordering and the associated map ρ_t , we now define the Compatibility Table. For fixed integers ℓ and r , consider the array

$$\mathcal{A} \in \{0, 1\}^{\binom{n}{\ell} \times \binom{n}{r}}.$$

The table is constructed as follows. For each ℓ -subset $S_\ell \subseteq [n]$, we compute its rank $i = \rho_\ell(S_\ell)$. For every r -subset $S_r \subseteq [n] \setminus S_\ell$, we compute $j = \rho_r(S_r)$ and set $\mathcal{A}[i][j] = 1$, while all remaining entries in row i are set to 0. Thus, the i -th row of $\mathcal{A}$ encodes precisely those r -subsets that are disjoint from S_ℓ , and therefore compatible with it.

To determine whether a pair of vectors $(\mathbf{s}_1, \mathbf{s}_2) \in \mathcal{S}_\ell(\mathbf{b}) \times \mathcal{S}_r(\mathbf{b})$ is compatible, we evaluate

$$i = \rho_\ell(\text{I-Supp}(\mathbf{s}_1)), \quad j = \rho_r(\text{I-Supp}(\mathbf{s}_2)),$$

and declare the pair compatible if and only if $\mathcal{A}[i][j] = 1$. An example of a Compatibility Table is provided in Appendix C.

The construction of the Compatibility Table $\mathcal{A}$ relies on efficient evaluation of the ranking functions ρ_ℓ and ρ_r . To this end, all relevant binomial coefficients are precomputed. With this preprocessing, a single evaluation of ρ_ℓ (resp. ρ_r) requires ℓ (resp. r) additions and subtractions on integers of bit-length $\log_2\binom{n}{\ell}$ (resp. $\log_2\binom{n}{r}$), which costs⁵ $\mathcal{O}(\ell \log_2\binom{n}{\ell})$ (resp. $\mathcal{O}(r \log_2\binom{n}{r})$).

The table $\mathcal{A}$ is generated using two nested loops. In the outer loop, we enumerate all ℓ -subsets $S_1 \subseteq [n]$. Rather than explicitly calling ρ_ℓ , we generate these subsets according to the defined lexicographic ordering and assign indices incrementally, incurring no additional ranking cost. In the inner loop, we enumerate all r -subsets of $[n] \setminus S_1$ and compute their ranks using ρ_r , which costs $\mathcal{O}(r \log_2\binom{n}{r})$ per call. Consequently, the overall cost of generating the Compatibility Table $\mathcal{A}$ is

$$\mathcal{O}\left(r \log_2\binom{n}{r} \cdot \binom{n}{\ell} \binom{n-\ell}{r}\right).$$

4.3 Algorithm for Solving PKP

We are now ready to present our algorithm for solving PKP instances. Recall that in order to solve Eq. (4), the KMP algorithm splits $\mathbf{x}_2^* = (\mathbf{s}_1^* \parallel \mathbf{s}_2^*)$ into two parts, with $\mathbf{s}_1^* \in \mathbb{F}_q^\ell$ and $\mathbf{s}_2^* \in \mathbb{F}_q^r$, and enumerates over all vectors in $\mathcal{S}_\ell(\mathbf{b})$ and $\mathcal{S}_r(\mathbf{b})$, respectively. Our method builds on this by introducing a constraint-based enumeration. For optimized values of μ_1 and μ_2 , we randomly select two disjoint subsets $A_1, A_2 \subseteq \text{Supp}(\mathbf{b})$ of size μ_1 and μ_2 , respectively and suppose that $A_1 \cup A_2 = A$. Since A_1 and A_2 are disjoint, the size of A becomes $\mu_1 + \mu_2$. Given these subsets, instead of exhaustively enumerating all $\mathbf{s}_1 \in \mathcal{S}_\ell(\mathbf{b})$ and $\mathbf{s}_2 \in \mathcal{S}_r(\mathbf{b})$ as in the KMP algorithm, we restrict the search to vectors $\mathbf{s}_1 \in \mathcal{S}_\ell(\mathbf{b})$ satisfying

$$A_1 \subseteq \text{Supp}(\mathbf{s}_1), \quad \text{and} \quad \text{Supp}(\mathbf{s}_1) \setminus A_1 \subseteq \text{Supp}(\mathbf{b}) \setminus A,$$

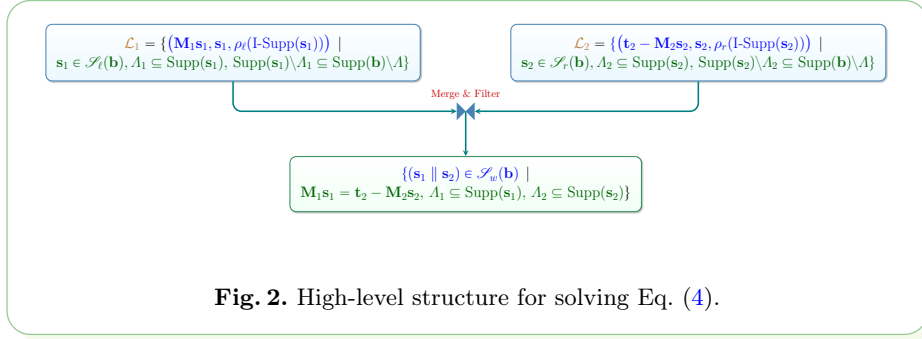
and vectors $\mathbf{s}_2 \in \mathcal{S}_r(\mathbf{b})$ satisfying

$$A_2 \subseteq \text{Supp}(\mathbf{s}_2), \quad \text{and} \quad \text{Supp}(\mathbf{s}_2) \setminus A_2 \subseteq \text{Supp}(\mathbf{b}) \setminus A.$$

Of course, we do not know in advance whether these subsets of $\text{Supp}(\mathbf{b})$ will actually appear in the support of $\mathbf{s}_1^*$ and $\mathbf{s}_2^*$. To address this uncertainty, we repeat the random selection and enumeration process multiple times until a valid solution is found.

The algorithm then proceeds analogously to the KMP algorithm by deriving Eq. (5). To solve this equation, the algorithm iterates over all $(\ell - \mu_1)$ -subsets $S_1 \subseteq \text{Supp}(\mathbf{b}) \setminus A$. For each such subset S_1 , the index $i = \rho_\ell(\{\delta \mid b_\delta \in S_1 \cup A_1\})$ is computed, and all vectors $\mathbf{s}_1$ with support equal to $S_1 \cup A_1$ are enumerated. This enumeration is performed using the SJT algorithm, which enables $\mathbf{M}_1 \mathbf{s}_1$ to

⁵This cost is a conservative upper bound, since integer bit operations are cheaper than field operations.



be updated incrementally as described in Section 4.1. For each generated vector $\mathbf{s}_1$, the tuple $(\mathbf{M}_1\mathbf{s}_1, \mathbf{s}_1, i)$ is stored in the list $\mathcal{L}_1$. Consequently,

$$\mathcal{L}_1 = \left\{ (\mathbf{M}_1\mathbf{s}_1, \mathbf{s}_1, \rho_\ell(\text{I-Supp}(\mathbf{s}_1))) \mid \mathbf{s}_1 \in \mathcal{S}_\ell(\mathbf{b}), A_1 \subseteq \text{Supp}(\mathbf{s}_1), \text{Supp}(\mathbf{s}_1) \setminus A_1 \subseteq \text{Supp}(\mathbf{b}) \setminus A \right\}.$$

Similarly, we iterate over all $(r - \mu_2)$ -subsets $S_2 \subseteq \text{Supp}(\mathbf{b}) \setminus A$ and enumerate all vectors $\mathbf{s}_2$ satisfying $\text{Supp}(\mathbf{s}_2) = S_2 \cup A_2$ via the SJT algorithm, and through incremental updates, we construct the list

$$\mathcal{L}_2 = \left\{ (\mathbf{t}_2 - \mathbf{M}_2\mathbf{s}_2, \mathbf{s}_2, \rho_r(\text{I-Supp}(\mathbf{s}_2))) \mid \mathbf{s}_2 \in \mathcal{S}_r(\mathbf{b}), A_2 \subseteq \text{Supp}(\mathbf{s}_2), \text{Supp}(\mathbf{s}_2) \setminus A_2 \subseteq \text{Supp}(\mathbf{b}) \setminus A \right\}.$$

Note that, as in KMP, the list $\mathcal{L}_1$ does not require storing the full tuples $(\mathbf{M}_1\mathbf{s}_1, \mathbf{s}_1, i)$. Instead, it suffices to store the pairs $(\mathbf{s}_1, i)$, using $\mathbf{M}_1\mathbf{s}_1$ solely as the hash key. Furthermore, the list $\mathcal{L}_2$ can be generated on the fly during the meet-in-the-middle phase.

To obtain a solution $\mathbf{x}_2$ to Eq. (4), we perform a meet-in-the-middle search between the lists $\mathcal{L}_1$ and $\mathcal{L}_2$. Whenever a collision occurs between an entry $(\mathbf{c}, \mathbf{s}_1, i) \in \mathcal{L}_1$ and an entry $(\mathbf{c}, \mathbf{s}_2, j) \in \mathcal{L}_2$, we query the Compatibility Table $\mathcal{A}[i][j]$. If $\mathcal{A}[i][j] = 1$, then $\mathbf{x}_2 = (\mathbf{s}_1 \parallel \mathbf{s}_2)$ is treated as a valid solution to Eq. (4). Consequently, after filtration we obtain the list

$$\{(\mathbf{s}_1 \parallel \mathbf{s}_2) \in \mathcal{S}_w(\mathbf{b}) \mid \mathbf{M}_1\mathbf{s}_1 = \mathbf{t}_2 - \mathbf{M}_2\mathbf{s}_2, A_1 \subseteq \text{Supp}(\mathbf{s}_1), A_2 \subseteq \text{Supp}(\mathbf{s}_2)\}.$$

For an illustration of how to find valid solutions to Eq. (4) see Figure 2.

We then compute the corresponding vector $\mathbf{x}_1$ via Eq. (3) for every obtained valid solution $\mathbf{x}_2$ of Eq. (4), and verify compatibility between $\mathbf{x}_1$ and $\mathbf{x}_2$. If the pair is compatible, the algorithm returns $(\mathbf{x}_1 \parallel \mathbf{x}_2)$ as the secret permutation vector solving the PKP instance. If no solution is obtained, the process restarts with a new random choice of (A_1, A_2) and repeats until a valid solution is found.

A pseudocode description of the algorithm is given in Algorithm 1.

Correctness of Algorithm 1. The repeated selection of the subsets A_1 and A_2 ensures that we will find a pair of subsets such that $A_1 \subseteq \text{Supp}(\mathbf{s}_1^*)$ and $A_2 \subseteq \text{Supp}(\mathbf{s}_2^*)$ holds.

Due to the compatibility condition, the remaining components of both $\mathbf{s}_1^*$ and $\mathbf{s}_2^*$ must lie within the set $\text{Supp}(\mathbf{b}) \setminus \Lambda$. To recover the full supports of $\mathbf{s}_1^*$ and $\mathbf{s}_2^*$, the algorithm enumerates over subsets of $\text{Supp}(\mathbf{b}) \setminus \Lambda$ of sizes $\ell - \mu_1$ and $r - \mu_2$, respectively.

Through this process and the collision search step, the algorithm successfully recovers the pair $(\mathbf{s}_1^* \parallel \mathbf{s}_2^*)$. With this, and Eq. (3), the algorithm reconstructs $\pi(\mathbf{b})$, thereby solving the PKP instance.

Complexity Analysis of Algorithm 1. We first account for the cost of the preprocessing phase, which consists of constructing the two `swap_sequences` corresponding to ℓ and r via the SJT algorithm, as well as the Compatibility Table $\mathcal{A} \in \{0, 1\}^{\binom{n}{\ell} \times \binom{n}{r}}$. As discussed in Sections 4.1 and 4.2, the total cost of constructing the `swap_sequences` is

$$\mathcal{O}(\ell \cdot \ell! + r \cdot r!),$$

while the cost of generating the Compatibility Table $\mathcal{A}$ is

$$\mathcal{O}\left(r \log_2\left(\binom{n}{r}\right) \cdot \binom{n}{\ell} \binom{n-\ell}{r}\right).$$

Therefore, the overall precomputation cost becomes

$$T_{\text{precompute}} = \mathcal{O}\left(\ell \cdot \ell! + r \cdot r! + r \log_2\left(\binom{n}{r}\right) \cdot \binom{n}{\ell} \binom{n-\ell}{r}\right).$$

The remaining time complexity of Algorithm 1 is determined by the number of repetitions required to succeed, multiplied by the cost of a single iteration. We first analyze the cost of a single iteration.

To construct $\mathcal{L}_1$, the support of any admissible vector $\mathbf{s}_1$ must contain the fixed subset Λ_1 , while the remaining $\ell - \mu_1$ positions are chosen from $\text{Supp}(\mathbf{b}) \setminus \Lambda$, which has cardinality $n - \mu_1 - \mu_2$. Hence, the total number of admissible supports is $\binom{n-\mu_1-\mu_2}{\ell-\mu_1}$. For each such support, we evaluate the rank function ρ_ℓ , which incurs a cost of $\mathcal{O}(\ell \log_2\left(\binom{n}{\ell}\right))$. Moreover, each fixed support gives rise to exactly $\ell!$ distinct vectors $\mathbf{s}_1$, which are enumerated using the precomputed `swap_sequence` corresponding to ℓ . The associated products $\mathbf{M}_1 \mathbf{s}_1$ are computed via incremental updates, which, as described in Section 4.1, incur a cost of $\mathcal{O}(k \cdot \ell!)$. Consequently, the cost associated with processing a single support is $\mathcal{O}(\ell \log_2\left(\binom{n}{\ell}\right) + k \cdot \ell!)$. Therefore, the total time complexity required to generate the list $\mathcal{L}_1$ is

$$T_{\mathcal{L}_1} = \mathcal{O}\left(\left(\ell \log_2\left(\binom{n}{\ell}\right) + k \cdot \ell!\right) \binom{n-\mu_1-\mu_2}{\ell-\mu_1}\right),$$

and the resulting size of $\mathcal{L}_1$ becomes $|\mathcal{L}_1| = \binom{n-\mu_1-\mu_2}{\ell-\mu_1} \ell!$.

An analogous argument applies to the construction of $\mathcal{L}_2$. Accordingly, the size of the list $\mathcal{L}_2$ is $|\mathcal{L}_2| = \binom{n-\mu_1-\mu_2}{r-\mu_2} r!$ and the total time complexity required to generate $\mathcal{L}_2$ is

$$T_{\mathcal{L}_2} = \mathcal{O}\left(\left(r \log_2\left(\binom{n}{r}\right) + k \cdot r!\right) \binom{n-\mu_1-\mu_2}{r-\mu_2}\right).$$

Algorithm 1: Algorithm for Solving PKP

Input: $(\tilde{\mathbf{A}}, \mathbf{b}) \in \mathbb{F}_q^{m \times n} \times \mathbb{F}_q^n$

- 1 Choose optimal k, ℓ , such that $1 \leq k \leq m, 0 \leq \ell \leq n - m + k$, and compute $w = n - m + k, r = w - \ell$;
- 2 Choose optimal μ_1 and μ_2 such that $0 \leq \mu_1 \leq \ell$ and $0 \leq \mu_2 \leq r$;
- 3 Find an invertible matrix $\mathbf{Q}$ such that $\mathbf{Q}\tilde{\mathbf{A}} = \begin{pmatrix} \mathbf{I}_{m-k} & \mathbf{M}' \\ \mathbf{0} & \mathbf{M} \end{pmatrix}$, and compute $\mathbf{t} = \tilde{\mathbf{Q}}(\sigma, 0, \dots, 0)^T, \sigma = \sum_i b_i$;
- 4 Set $\mathbf{M}_1 \leftarrow \mathbf{M}[:, \ell], \mathbf{M}_2 \leftarrow \mathbf{M}[\ell :, :], \mathbf{t}_1 \leftarrow \mathbf{t}[:, m - k]$, and $\mathbf{t}_2 \leftarrow \mathbf{t}[m - k :];$
// Compute Compatibility Table
- 5 Set $\mathcal{A} \leftarrow \mathbf{0} \in \{0, 1\}^{\binom{n}{\ell} \times \binom{n}{r}}$;
- 6 **for** $S_1 \subseteq [n]$ of size ℓ **do**
- 7 $i \leftarrow \rho_\ell(S_1)$;
- 8 **for** $S_2 \subseteq [n] \setminus S_1$ of size r **do**
- 9 $j \leftarrow \rho_r(S_2)$;
- 10 $\mathcal{A}[i][j] \leftarrow 1$;
- 11 **repeat**
- 12 Choose disjoint random subsets $A_1, A_2 \subseteq \text{Supp}(\mathbf{b})$ such that $|A_1| = \mu_1$ and $|A_2| = \mu_2$;
- 13 Compute $A \leftarrow A_1 \cup A_2$;
- 14 *// Lists $\mathcal{L}_1$ and $\mathcal{L}_2$ are generated via SJT-based incremental updates (Section 4.1), using hash maps keyed by $\mathbf{M}_1 \mathbf{s}_1$ and $\mathbf{t}_2 - \mathbf{M}_2 \mathbf{s}_2$, respectively.*
- 14 Generate $\mathcal{L}_1 \leftarrow \left\{ (\mathbf{M}_1 \mathbf{s}_1, \mathbf{s}_1, \rho_\ell(\text{I-Supp}(\mathbf{s}_1))) \mid \mathbf{s}_1 \in \mathcal{S}_\ell(\mathbf{b}), A_1 \subseteq \text{Supp}(\mathbf{s}_1), \text{Supp}(\mathbf{s}_1) \setminus A_1 \subseteq \text{Supp}(\mathbf{b}) \setminus A \right\}$;
- 15 Generate $\mathcal{L}_2 \leftarrow \left\{ (\mathbf{t}_2 - \mathbf{M}_2 \mathbf{s}_2, \mathbf{s}_2, \rho_r(\text{I-Supp}(\mathbf{s}_2))) \mid \mathbf{s}_2 \in \mathcal{S}_r(\mathbf{b}), A_2 \subseteq \text{Supp}(\mathbf{s}_2), \text{Supp}(\mathbf{s}_2) \setminus A_2 \subseteq \text{Supp}(\mathbf{b}) \setminus A \right\}$;
- 16 *// Merge via meet-in-the-middle and check for compatibility*
- 16 **for** $(\mathbf{c}, \mathbf{s}_1, i) \in \mathcal{L}_1$ and $(\mathbf{d}, \mathbf{s}_2, j) \in \mathcal{L}_2$ **do**
- 17 **if** $\mathbf{c} = \mathbf{d}$ and $\mathcal{A}[i][j] = 1$ **then**
- 18 $\mathbf{x}_2 \leftarrow (\mathbf{s}_1 \parallel \mathbf{s}_2)$;
- 19 compute $\mathbf{x}_1 \leftarrow \mathbf{t}_1 - \mathbf{M}' \mathbf{x}_2$;
- 20 **if** $(\mathbf{x}_1 \parallel \mathbf{x}_2) \in \mathcal{S}_n(\mathbf{b})$ *// Check compatibility via intersection*
- 21 **then**
- 22 **return** $(\mathbf{x}_1 \parallel \mathbf{x}_2)$;
- 23 **until** Solution found;

Since Eq. (4) consists of k independent equations, the expected number of collisions between $\mathcal{L}_1$ and $\mathcal{L}_2$ is $\frac{|\mathcal{L}_1||\mathcal{L}_2|}{q^k}$. For each collision, we must check the

compatibility using the Compatibility Table $\mathcal{A}$, so the cost of this step becomes

$$\mathcal{O}\left(\frac{|\mathcal{L}_1||\mathcal{L}_2|}{q^k}\right).$$

Among these collisions the filtered vectors that survive the compatibility test are the valid solutions of Eq. (4). Therefore, the expected number of filtered vectors that survive the compatibility test is

$$N_{\text{filtered}} = \frac{\binom{n-\mu_1-\mu_2}{\ell-\mu_1} \ell! \cdot \binom{n-\ell-\mu_2}{r-\mu_2} r!}{q^k}.$$

For each such filtered vector $\mathbf{x}_2$, we compute $\mathbf{x}_1 = \mathbf{t}_1 - \mathbf{M}'\mathbf{x}_2$ and verify the compatibility of $(\mathbf{x}_1, \mathbf{x}_2)$. These operations cost

$$\mathcal{O}((m-k)w + \min\{m-k, w\}) \cdot N_{\text{filtered}}.$$

Combining the above contributions, the total cost of a single iteration of the algorithm becomes

$$T_{\text{iter}} = \mathcal{O}\left(T_{\mathcal{L}_1} + T_{\mathcal{L}_2} + \frac{|\mathcal{L}_1||\mathcal{L}_2|}{q^k} + ((m-k)w + \min\{m-k, w\}) \cdot N_{\text{filtered}}\right).$$

To determine the number of repetitions needed for success, observe that a random pair of disjoint subsets (A_1, A_2) of $\text{Supp}(\mathbf{b})$ satisfies $A_1 \subseteq \text{Supp}(\mathbf{s}_1^*)$ and $A_2 \subseteq \text{Supp}(\mathbf{s}_2^*)$ with probability

$$p = \frac{\binom{\ell}{\mu_1} \binom{r}{\mu_2}}{\binom{n}{\mu_1} \binom{n-\mu_1}{\mu_2}},$$

so the expected number of repetitions is $1/p$, and the overall cost becomes

$$T_{\text{Alg-1}} = \mathcal{O}\left(T_{\text{precompute}} + \frac{1}{p} \cdot T_{\text{iter}}\right).$$

Finally, we describe the memory complexity of the algorithm. Only the list $\mathcal{L}_1$ must be stored explicitly, whereas the lists $\mathcal{L}_2$ can be generated on the fly. Thus, the memory usage is determined by the list $\mathcal{L}_1$ together with the swap_sequences and the Compatibility Table $\mathcal{A}$. As in KMP, it suffices for $\mathcal{L}_1$ to store the pairs $(\mathbf{s}_1, \rho_\ell(\text{I-Supp}(\mathbf{s}_1)))$, using a hash map keyed by $\mathbf{M}_1\mathbf{s}_1$. The cost of hashing is negligible and does not affect the overall complexity.

We measure memory usage in bits. The swap_sequences store swap positions bounded by ℓ and r , respectively, and therefore require $\ell! \cdot \log_2(\ell) + r! \cdot \log_2(r)$ bits of storage. The Compatibility Table $\mathcal{A}$ is binary and thus requires $\binom{n}{\ell} \cdot \binom{n}{r}$ bits. Each vector $\mathbf{s}_1$ from $\mathcal{L}_1$ has length ℓ with entries in $\mathbb{F}_q$, so storing all ℓ entries directly requires $\ell \log_2(q)$ bits. For storage efficiency, however, instead of

keeping the full vector $\mathbf{s}_1$, we may store only its ordered I-Supp⁶. Representing $\mathbf{s}_1$ in this way requires only $\ell \log_2(n)$ bits, which is strictly smaller than the storage needed for the full vector. Thus the size of the stored pair $(\mathbf{s}_1, \rho_\ell(\text{I-Supp}(\mathbf{s}_1)))$ becomes $\ell \cdot \log_2(n) + \log_2 \left(\binom{n}{\ell} \right)$ bits. Therefore, the total memory requirement of the algorithm, measured in bits, is

$$M_{\text{Alg-1}} = \mathcal{O} \left(\ell! \cdot \log_2(\ell) + r! \cdot \log_2(r) + \binom{n}{\ell} \cdot \binom{n}{r} + (\ell \cdot \log_2(n) + \log_2 \left(\binom{n}{\ell} \right)) \cdot |\mathcal{L}_1| \right)$$

Remark 2. In the asymptotic regime, Algorithm 1 does not provide an improvement over the KMP algorithm. A detailed analysis is given in Appendix D.

Remark 3. As discussed in Section 3.1, the memory usage of the KMP algorithm consists of storing a list of ℓ -vectors of size ${}^nP_\ell$. To compare this with the memory requirements of our proposed algorithm, we express this cost in bits too. Each ℓ -vector can be stored via its ordered I-Supp, requiring $\ell \log_2(n)$ bits. Hence, the memory complexity (in bits) of the KMP algorithm becomes

$$M_{\text{KMP}} = \mathcal{O} \left(\ell \log_2(n) \cdot {}^nP_\ell \right).$$

4.4 Adaptation of Our Approach to the SBC Algorithm

The same strategy can also be applied to the SBC algorithm, leading to improved complexity compared to the original SBC method. However, across most parameter choices, and in particular within the cryptographically relevant regime (see Table 2), this adaptation does not outperform Algorithm 1. For this reason, we present the detailed analysis in Appendix H.

5 A Time-Memory trade-off using Schroeppe–Shamir Adaptation for PKP

All existing attacks on PKP-based schemes have primarily focused on reducing the time complexity. Nevertheless, both the KMP and SBC algorithms still require very large amounts of memory. This naturally motivates the design of memory-efficient approaches, notably by adapting the classical Schroeppe–Shamir technique. In this section, we present a time–memory trade-off based on such an adaptation, built upon Algorithm 1, for solving the PKP problem.

In analogy with Algorithm 1, we perform a constraint-based enumeration over the vectors $\mathbf{s}_1$ and $\mathbf{s}_2$ to solve Eq. (5). For optimized values $0 \leq \mu_1 \leq \ell$

⁶Note that I-Supp identifies only the underlying index set and therefore corresponds to multiple vectors. However, once an ordering of the elements in the I-Supp set is fixed, this ordered index set determines a unique vector. For example, let $\mathbf{b} = (3, 2, 7)^T$. The unordered I-Supp set $\{0, 1\}$ corresponds to the vectors $(3, 2)^T$ and $(2, 3)^T$, whereas the ordered I-Supp $(0, 1)$ and $(1, 0)$ correspond uniquely to $(3, 2)^T$ and $(2, 3)^T$, respectively. Here and throughout the paper, an *ordered* I-Supp is therefore understood as an ordered tuple of indices.

and $0 \leq \mu_2 \leq r$, we randomly choose disjoint subsets $\Lambda_1, \Lambda_2 \subseteq \text{Supp}(\mathbf{b})$ with $|\Lambda_i| = \mu_i$. We then constrain the vectors $\mathbf{s}_1$ and $\mathbf{s}_2$ by enforcing

$$\Lambda_1 \subseteq \text{Supp}(\mathbf{s}_1), \quad \text{Supp}(\mathbf{s}_1) \setminus \Lambda_1 \subseteq \text{Supp}(\mathbf{b}) \setminus \Lambda,$$

and

$$\Lambda_2 \subseteq \text{Supp}(\mathbf{s}_2), \quad \text{Supp}(\mathbf{s}_2) \setminus \Lambda_2 \subseteq \text{Supp}(\mathbf{b}) \setminus \Lambda,$$

where, $\Lambda = \Lambda_1 \cup \Lambda_2$. We sample such random pairs (Λ_1, Λ_2) until a valid solution for the PKP instance is found. A single iteration proceeds as follows.

To solve Eq. (5), Algorithm 1 performs a meet-in-the-middle search using constraint-based enumeration. The Schroeppe-Shamir method accomplishes the same task using a 4-way merge. For this, we split both vectors as

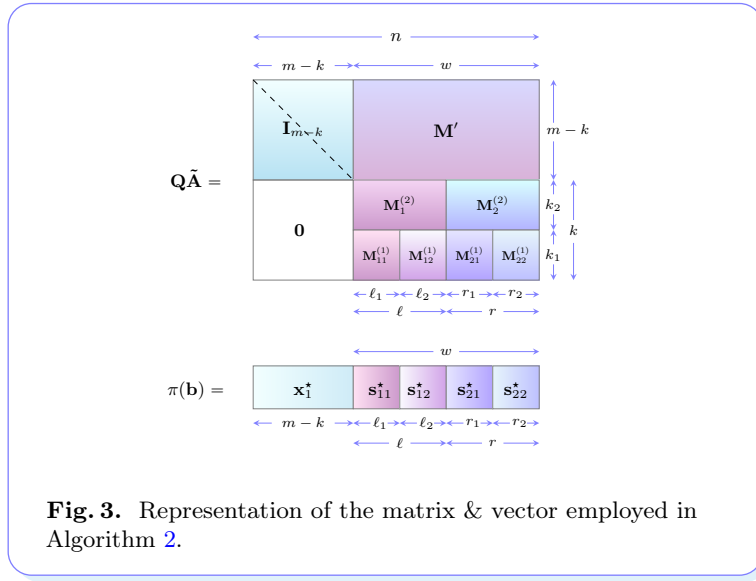
$$\mathbf{s}_1 = (\mathbf{s}_{11} \parallel \mathbf{s}_{12}) \in \mathbb{F}_q^{\ell_1} \times \mathbb{F}_q^{\ell_2}, \quad \mathbf{s}_2 = (\mathbf{s}_{21} \parallel \mathbf{s}_{22}) \in \mathbb{F}_q^{r_1} \times \mathbb{F}_q^{r_2},$$

where ℓ_1 and r_1 are optimization parameters, and $\ell_2 = \ell - \ell_1$ and $r_2 = r - r_1$.

Next, we project Eq. (5) onto the lower k_1 coordinates, where k_1 is another optimization parameter. This yields the relation

$$\mathbf{M}_{11}^{(1)} \mathbf{s}_{11} + \mathbf{M}_{12}^{(1)} \mathbf{s}_{12} = \mathbf{t}_2^{(1)} - \mathbf{M}_{21}^{(1)} \mathbf{s}_{21} - \mathbf{M}_{22}^{(1)} \mathbf{s}_{22}. \quad (5.a)$$

Here, $\mathbf{t}_2$ is split as $(\mathbf{t}_2^{(2)} \parallel \mathbf{t}_2^{(1)})$, with $\mathbf{t}_2^{(1)} \in \mathbb{F}_q^{k_1}$. The matrix and vector structures involved are shown in Figure 3.



To apply the Schroeppe-Shamir technique, we introduce an intermediate vector $\mathbf{r} \in \mathbb{F}_q^{k_1}$. We first search for vectors satisfying

$$\mathbf{M}_{11}^{(1)} \mathbf{s}_{11} + \mathbf{M}_{12}^{(1)} \mathbf{s}_{12} = \mathbf{r}, \quad \text{and} \quad \mathbf{M}_{21}^{(1)} \mathbf{s}_{21} + \mathbf{M}_{22}^{(1)} \mathbf{s}_{22} = \mathbf{t}_2^{(1)} - \mathbf{r}.$$

These equations can be rewritten as

$$\mathbf{M}_{11}^{(1)} \mathbf{s}_{11} = \mathbf{r} - \mathbf{M}_{12}^{(1)} \mathbf{s}_{12} \quad (5.b)$$

and

$$\mathbf{M}_{21}^{(1)} \mathbf{s}_{21} = \mathbf{t}_2^{(1)} - \mathbf{r} - \mathbf{M}_{22}^{(1)} \mathbf{s}_{22} \quad (5.c)$$

Consider to solve Eq. (5.b). If $\Lambda_1 \subseteq \text{Supp}(\mathbf{s}_1^*)$, the support of $\mathbf{s}_1^*$ must distribute the elements of Λ_1 between the supports of $\mathbf{s}_{11}^*$ and $\mathbf{s}_{12}^*$. Thus there exists a subset $\Lambda_{11} \subseteq \Lambda_1$ such that

$$\Lambda_{11} \subseteq \text{Supp}(\mathbf{s}_{11}^*) \quad \text{and} \quad \Lambda_{12} := \Lambda_1 \setminus \Lambda_{11} \subseteq \text{Supp}(\mathbf{s}_{12}^*).$$

Hence, we iterate over all subsets $\Lambda_{11} \subseteq \Lambda_1$ satisfying

$$|\Lambda_{11}| \leq \ell_1 \quad \text{and} \quad |\Lambda_{12}| = \mu_1 - |\Lambda_{11}| \leq \ell_2, \quad (C1)$$

so that both $\mathbf{s}_{11}$ and $\mathbf{s}_{12}$ can accommodate the required fixed positions. For each such choice, we use the SJT algorithm described in Section 4.1 to generate the lists

$$\mathcal{L}_{\Lambda_{11}}^{(1)} = \left\{ \left(\mathbf{M}_{11}^{(1)} \mathbf{s}_{11}, \mathbf{s}_{11} \right) \mid \mathbf{s}_{11} \in \mathcal{S}_{\ell_1}(\mathbf{b}), \Lambda_{11} \subseteq \text{Supp}(\mathbf{s}_{11}), \text{Supp}(\mathbf{s}_{11}) \setminus \Lambda_{11} \subseteq \text{Supp}(\mathbf{b}) \setminus \Lambda \right\},$$

$$\mathcal{L}_{\Lambda_{12}}^{(1)} = \left\{ \left(\mathbf{r} - \mathbf{M}_{12}^{(1)} \mathbf{s}_{12}, \mathbf{s}_{12} \right) \mid \mathbf{s}_{12} \in \mathcal{S}_{\ell_2}(\mathbf{b}), \Lambda_{12} \subseteq \text{Supp}(\mathbf{s}_{12}), \text{Supp}(\mathbf{s}_{12}) \setminus \Lambda_{12} \subseteq \text{Supp}(\mathbf{b}) \setminus \Lambda \right\}.$$

We perform a meet-in-the-middle collision search on these lists, followed by a compatibility filtering step, to obtain

$$\begin{aligned} \mathcal{L}_{\Lambda_{11}} &= \left\{ \left(\mathbf{M}_1^{(2)}(\mathbf{s}_{11} \parallel \mathbf{s}_{12}), (\mathbf{s}_{11} \parallel \mathbf{s}_{12}) \right) \mid (\mathbf{s}_{11} \parallel \mathbf{s}_{12}) \in \mathcal{S}_\ell(\mathbf{b}), \mathbf{M}_{11}^{(1)} \mathbf{s}_{11} = \mathbf{r} - \mathbf{M}_{12}^{(1)} \mathbf{s}_{12}, \right. \\ &\quad \left. \Lambda_{11} \subseteq \text{Supp}(\mathbf{s}_{11}), \Lambda_{12} \subseteq \text{Supp}(\mathbf{s}_{12}), \text{Supp}((\mathbf{s}_{11} \parallel \mathbf{s}_{12})) \subseteq \text{Supp}(\mathbf{b}) \setminus \Lambda \right\}. \end{aligned}$$

Taking union over all such subsets Λ_{11} , we obtain the set

$$\mathcal{L}_1 = \bigcup_{\substack{\Lambda_{11} \subseteq \Lambda_1 \\ |\Lambda_{11}| \leq \ell_1 \\ \ell - |\Lambda_{11}| \leq \ell_2}} \mathcal{L}_{\Lambda_{11}}.$$

For distinct choices of Λ_{11} , the lists $\mathcal{L}_{\Lambda_{11}}$ are disjoint and together exhaust the valid solution space of Eq. (5.b). Hence, by construction, $\mathcal{L}_1$ contains exactly all valid solutions of Eq. (5.b), that is,

$$\begin{aligned} \mathcal{L}_1 &= \left\{ \left(\mathbf{M}_1^{(2)}(\mathbf{s}_{11} \parallel \mathbf{s}_{12}), (\mathbf{s}_{11} \parallel \mathbf{s}_{12}) \right) \mid (\mathbf{s}_{11} \parallel \mathbf{s}_{12}) \in \mathcal{S}_\ell(\mathbf{b}), \mathbf{M}_{11}^{(1)} \mathbf{s}_{11} = \mathbf{r} - \mathbf{M}_{12}^{(1)} \mathbf{s}_{12}, \right. \\ &\quad \left. \Lambda_1 \subseteq \text{Supp}((\mathbf{s}_{11} \parallel \mathbf{s}_{12})), \text{Supp}((\mathbf{s}_{11} \parallel \mathbf{s}_{12})) \subseteq \text{Supp}(\mathbf{b}) \setminus \Lambda \right\}. \end{aligned}$$

A similar construction is applied to Eq. (5.c). By iterating over all subsets $\Lambda_{21} \subseteq \Lambda_2$ satisfying

$$|\Lambda_{21}| \leq r_1 \quad \text{and} \quad \mu_2 - |\Lambda_{21}| \leq r_2,$$

we obtain the list

$$\mathcal{L}_2 = \left\{ \left(\mathbf{t}_2^{(2)} - \mathbf{M}_2^{(2)}(\mathbf{s}_{21} \parallel \mathbf{s}_{22}), (\mathbf{s}_{21} \parallel \mathbf{s}_{22}) \right) \mid (\mathbf{s}_{21} \parallel \mathbf{s}_{22}) \in \mathcal{S}_r(\mathbf{b}), \mathbf{M}_{21}^{(1)} \mathbf{s}_{21} = \mathbf{t}_2^{(1)} - \mathbf{r} - \mathbf{M}_{22}^{(1)} \mathbf{s}_{22}, \right. \\ \left. A_2 \subseteq \text{Supp}(\mathbf{s}_{21} \parallel \mathbf{s}_{22}), \text{Supp}((\mathbf{s}_{21} \parallel \mathbf{s}_{22})) \subseteq \text{Supp}(\mathbf{b}) \setminus A \right\}.$$

Observe that for any $(\mathbf{c}, \mathbf{s}_1) \in \mathcal{L}_1$ and $(\mathbf{d}, \mathbf{s}_2) \in \mathcal{L}_2$, the concatenated vector $(\mathbf{s}_1 \parallel \mathbf{s}_2)$ already satisfies Eq. (5) on the lower k_1 coordinates. By performing a meet-in-the-middle procedure on the upper k_2 coordinates, where $k_2 = k - k_1$, we obtain vectors that satisfy Eq. (5) in its entirety. Accordingly, we merge the lists $\mathcal{L}_1$ and $\mathcal{L}_2$ and apply a compatibility filtration. An illustration of this four-way merge is given in Figure 4. For each resulting vector $\mathbf{x}_2$, we compute $\mathbf{x}_1$ via Eq. (3) and subsequently verify whether $(\mathbf{x}_1 \parallel \mathbf{x}_2)$ is compatible. If so, the corresponding vector is returned as a valid solution to the PKP instance. This procedure is repeated for all $\mathbf{r} \in \mathbb{F}_q^{k_1}$ until a solution is found.

The complete pseudocode is provided in Algorithm 2 in Appendix E.

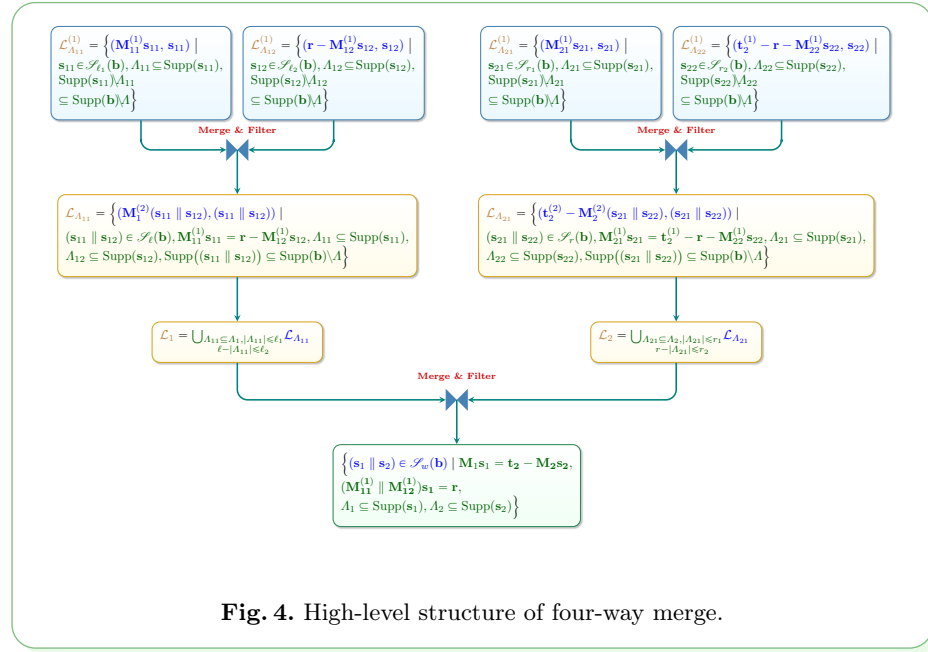


Fig. 4. High-level structure of four-way merge.

Complexity Analysis of Algorithm 2. In the precomputation phase, we store the swap_sequences corresponding to the integers ℓ_1, ℓ_2, r_1 , and r_2 , generated using the SJT algorithm. These sequences are subsequently used to generate the lists $\mathcal{L}_{A_{ij}}^{(1)}$. Consequently, the time complexity of the precomputation phase is

$$T_{\text{precompute}} = \mathcal{O}(\ell_1 \cdot \ell_1! + \ell_2 \cdot \ell_2! + r_1 \cdot r_1! + r_2 \cdot r_2!).$$

The memory required to store these `swap_sequences` (in bits) is given by

$$M_{\text{precompute}} = \mathcal{O}(\log_2(\ell_1) \cdot \ell_1! + \log_2(\ell_2) \cdot \ell_2! + \log_2(r_1) \cdot r_1! + \log_2(r_2) \cdot r_2!).$$

The remaining time complexity of Algorithm 2 is determined by two nested components:

- (i) the outer repeat loop, which randomly selects pairs of disjoint subsets (A_1, A_2) ,
- (ii) the inner loop over all intermediate constraints $\mathbf{r} \in \mathbb{F}_q^{k_1}$.

Hence, the overall time complexity equals the cost of a single iteration multiplied by q^{k_1} and by the expected number of repetitions required to obtain a valid pair of subsets. We therefore begin by analyzing the cost of one iteration.

Fix $|A_{11}| = i$. From the constraint (C1) on A_{11} , we have

$$\max\{0, \mu_1 - \ell_2\} \leq i \leq \min\{\mu_1, \ell_1\}.$$

The list $\mathcal{L}_{A_{11}}^{(1)}$ has size $\binom{n-\mu_1-\mu_2}{\ell_1-i} \ell_1!$, and its construction incurs a cost of $\mathcal{O}\left(k_1 |\mathcal{L}_{A_{11}}^{(1)}|\right)$ time using SJT-based incremental updates. Likewise, the list $\mathcal{L}_{A_{12}}^{(1)}$ has size $\binom{n-\mu_1-\mu_2}{\ell_2-(\mu_1-i)} \ell_2!$, and its construction cost is $\mathcal{O}\left(k_1 |\mathcal{L}_{A_{12}}^{(1)}|\right)$. To obtain the list $\mathcal{L}_{A_{11}}$, we perform a collision search between $\mathcal{L}_{A_{11}}^{(1)}$ and $\mathcal{L}_{A_{12}}^{(1)}$, followed by compatibility filtering. This incurs a cost of

$$\mathcal{O}\left(\min\{\ell_1, \ell_2\} \frac{|\mathcal{L}_{A_{11}}^{(1)}| |\mathcal{L}_{A_{12}}^{(1)}|}{q^{k_1}}\right).$$

Let T_{1i} denote the total complexity up to this stage to obtain $\mathcal{L}_{A_{11}}$. Then

$$T_{1i} = \mathcal{O}\left(k_1 \binom{n-\mu_1-\mu_2}{\ell_1-i} \ell_1! + k_1 \binom{n-\mu_1-\mu_2}{\ell_2-(\mu_1-i)} \ell_2! + \min\{\ell_1, \ell_2\} \frac{\binom{n-\mu_1-\mu_2}{\ell_1-i} \ell_1! \binom{n-\mu_1-\mu_2}{\ell_2-(\mu_1-i)} \ell_2!}{q^{k_1}}\right).$$

After filtering, each surviving vector requires one additional matrix-vector multiplication to produce an element of $\mathcal{L}_{A_{11}}$. Over all valid subsets A_{11} of A_1 , the expected number of filtered vectors, and thus the expected size of $\mathcal{L}_1$, is $|\mathcal{L}_1| = \frac{\binom{n-\mu_1-\mu_2}{\ell-\mu_1} \ell!}{q^{k_1}}$. Therefore, the total cost for constructing $\mathcal{L}_1$ is

$$T_{\mathcal{L}_1} = \mathcal{O}\left(\sum_{\max\{0, \mu_1 - \ell_2\} \leq i \leq \min\{\mu_1, \ell_1\}} \binom{\mu_1}{i} T_{1i} + k_2 \ell |\mathcal{L}_1|\right).$$

A symmetric argument yields the expected size of $\mathcal{L}_2$, namely $|\mathcal{L}_2| = \frac{\binom{n-\mu_1-\mu_2}{r-\mu_2} r!}{q^{k_1}}$, and its total generation cost

$$T_{\mathcal{L}_2} = \mathcal{O}\left(\sum_{\max\{0, \mu_2 - r_2\} \leq i \leq \min\{\mu_2, r_1\}} \binom{\mu_2}{i} T_{2i} + k_2 r |\mathcal{L}_2|\right),$$

where

$$T_{2i} = \mathcal{O}\left(k_1 \binom{n-\mu_1-\mu_2}{r_1-i} r_1! + k_1 \binom{n-\mu_1-\mu_2}{r_2-(\mu_2-i)} r_2! + \min\{r_1, r_2\} \frac{\binom{n-\mu_1-\mu_2}{r_1-i} r_1! \binom{n-\mu_1-\mu_2}{r_2-(\mu_2-i)} r_2!}{q^{k_1}}\right).$$

To recover the solutions of Eq. (5), we then merge $\mathcal{L}_1$ and $\mathcal{L}_2$ using a meet-in-the-middle procedure on the upper k_2 coordinates and apply compatibility filtering. This step has cost

$$\mathcal{O}\left(\min\{\ell, r\} \frac{|\mathcal{L}_1| |\mathcal{L}_2|}{q^{k_2}}\right).$$

The expected number of filtered vectors, equivalently the number of valid common solutions to Eq. (5) and Eq. (5.b), is

$$N_{\text{filtered}} = \frac{\binom{n-\mu_1-\mu_2}{\ell-\mu_1} \ell! \binom{n-\ell-\mu_2}{r-\mu_2} r!}{q^{k+k_1}}.$$

For each such vector, we compute $\mathbf{x}_1$ and verify compatibility, costing

$$\mathcal{O}\left((w(m-k) + \min\{w, m-k\}) N_{\text{filtered}}\right).$$

Thus, the total time for a single iteration of the algorithm is

$$T_{\text{iter}} = \mathcal{O}\left(T_{\mathcal{L}_1} + T_{\mathcal{L}_2} + \min\{\ell, r\} \frac{|\mathcal{L}_1| |\mathcal{L}_2|}{q^{k_2}} + (w(m-k) + \min\{w, m-k\}) N_{\text{filtered}}\right).$$

Finally, as discussed in Section 4.3, the probability that a randomly selected pair (A_1, A_2) is valid equals

$$p = \frac{\binom{\ell}{\mu_1} \binom{r}{\mu_2}}{\binom{n}{\mu_1} \binom{n-\mu_1}{\mu_2}}.$$

Therefore, the overall time complexity of Algorithm 2 is

$$T_{\text{Alg-2}} = \mathcal{O}\left(T_{\text{precompute}} + \frac{q^{k_1}}{p} \cdot T_{\text{iter}}\right).$$

Now let us analyze the memory complexity of Algorithm 2. During execution, the algorithm must store the list $\mathcal{L}_1$, and, in order to construct the lists $\mathcal{L}_1$ and $\mathcal{L}_2$, it must also store the lists $\mathcal{L}_{A_{11}}^{(1)}$ and $\mathcal{L}_{A_{21}}^{(1)}$. All remaining lists can be generated on the fly and therefore do not contribute to the memory usage. Thus, the total memory required for generating $\mathcal{L}_1$ and $\mathcal{L}_2$ is determined by the maximum memory requirement of $\mathcal{L}_{A_{11}}^{(1)}$ and $\mathcal{L}_{A_{21}}^{(1)}$ over all valid subsets.

For each element of the form $(\mathbf{M}_{11}^{(1)} \mathbf{s}_{11}, \mathbf{s}_{11})$ in $\mathcal{L}_{A_{11}}^{(1)}$, we only need to store the ordered I-Supp of the vector $\mathbf{s}_{11}$, since the value $\mathbf{M}_{11}^{(1)} \mathbf{s}_{11}$ is used solely for hashing in the meet-in-the-middle step. Therefore, the memory required for $\mathcal{L}_{A_{11}}^{(1)}$ is $\mathcal{O}(\ell_1 \log_2(n) |\mathcal{L}_{A_{11}}^{(1)}|)$. Similarly, storing the vectors in $\mathcal{L}_{A_{21}}^{(1)}$ requires

$\mathcal{O}\left(r_1 \log_2(n) |\mathcal{L}_{A_{21}}^{(1)}|\right)$. Thus, the memory required during the generation of $\mathcal{L}_1$ and $\mathcal{L}_2$ is

$$\mathcal{O}\left(\max\left\{\max_{A_{11}}\{\ell_1 \log_2(n) |\mathcal{L}_{A_{11}}^{(1)}|\}, \max_{A_{21}}\{r_1 \log_2(n) |\mathcal{L}_{A_{21}}^{(1)}|\}\right\}\right).$$

In addition, storing $\mathcal{L}_1$ itself requires $\mathcal{O}(\ell \log_2(n) |\mathcal{L}_1|)$. Combining these contributions, the total memory complexity of Algorithm 2 is

$$M_{\text{Alg-2}} = \mathcal{O}\left(M_{\text{precompute}} + \max\left\{\max_{A_{11}}\{\ell_1 \log_2(n) |\mathcal{L}_{A_{11}}^{(1)}|\}, \max_{A_{21}}\{r_1 \log_2(n) |\mathcal{L}_{A_{21}}^{(1)}|\}\right\} + \ell \log_2(n) |\mathcal{L}_1|\right).$$

Remark 4. Observe that, we have not included Compatibility Tables in either of the filtering stages. This is because, for the parameter sets used in Table 3, the size of the Compatibility Tables exceeds the memory requirements derived above. However, for suitable parameter choices, Compatibility Tables may be incorporated into the filtration phase to further reduce the time complexity.

Remark 5. All proposed algorithms in this work are described for the homogeneous PKP. However, our methods and the existing KMP and SBC algorithms extend naturally to the inhomogeneous version of PKP. This is because these algorithms do not rely on any specific structural property of the target vector. Consequently, the same algorithms and complexity bounds hold in the inhomogeneous setting.

6 Numerical Results

To assess the effectiveness of our proposed algorithms in the cryptanalysis of PKP-based schemes, we evaluate their performance on the parameter sets of PERK v2.2.0 [1], SUSHSYFISH [8], and PKP-DSS [9]. These parameter sets correspond to three security levels, which we denote as “Scheme I”, “Scheme II”, and “Scheme III” (e.g., PERK I, PERK II, PERK III). In the previous sections, time complexity is expressed in field-operation counts throughout the analysis of all algorithms. Following the convention of [13], we report in this section the bit-level time complexity by scaling the field-operation count T_{alg} by $\log_2(q)$; that is, the bit-complexity equals $\log_2(q) T_{\text{alg}}$.

6.1 Results for Algorithms 1 and 3

We present a comparative analysis of the time and memory bit-complexity required to recover the secret vector of a PKP instance using KMP, SBC, and our proposed methods (Algorithms 1 and 3). The corresponding bit-complexity exponents for the PERK v2.2.0, SUSHSYFISH and PKP-DSS parameter sets are summarized in Table 2.

From the comparison in Table 2, we observe that our proposed approaches, in particular Algorithm 1, achieve lower time bit-complexity in all cases and lower memory bit-complexity in almost all cases compared to the previously known

Table 2. Time and Memory bit-complexity exponents for the KMP, SBC, and our algorithms (Algorithm 1 and 3) when applied to the PERK v2.2.0, SUSHSYFISH and PKP-DSS parameter sets.

Schemes	n, m, q	KMP [20]		SBC [26]		Algorithm 1		Algorithm 3	
		Time	Mem	Time	Mem	Time	Mem	Time	Mem
PERK v2.2.0	64, 28, 2048	150.60	144.01	150.60	144.01	145.94	143.08	145.94	143.81
	92, 44, 2048	222.32	213.55	222.32	213.55	216.49	210.43	216.57	212.07
	118, 60, 2048	286.88	278.91	286.88	278.91	281.32	277.24	281.36	278.75
SUSHSYFISH	61, 29, 997	137.64	131.25	137.64	131.25	133.18	130.28	133.18	131.11
	87, 43, 1409	204.66	198.56	204.66	198.56	199.69	190.02	199.70	199.04
	111, 56, 1889	268.60	262.04	268.60	262.04	263.31	262.50	263.31	262.50
PKP-DSS	69, 42, 251	136.22	130.38	134.78	124.69	131.87	130.92	131.16	122.44
	94, 55, 509	202.09	190.68	199.24	190.68	196.31	184.51	198.49	173.24
	106, 48, 4093	270.21	258.83	270.21	258.83	262.75	253.00	262.94	255.89

methods. In particular, for the PERK v2.2.0 parameter sets, which are currently under consideration in the NIST process, the secret vector corresponding to the 150-bit, 220-bit, and 286-bit estimated bit-security levels([1], Section 7.2.5) can be recovered in time $2^{145.94}$, $2^{216.49}$, and $2^{281.32}$, respectively. This indicates that the effective security margins of these parameter sets are slightly smaller under our analysis. For the SUSHSYFISH and PKP-DSS parameter sets, our algorithms likewise yield improved bit-complexity compared to existing approaches.

Figure 5 further illustrates the trade-off between memory and time bit-complexity exponents by comparing our algorithms with the KMP and SBC complexities. Since Algorithm 3 improves upon Algorithm 1 only for the PKP-DSS I parameter set, the figure reports complexity exponents for this case alone. The plot shows that, when the time complexity of our method is increased to match that of KMP, a double-digit reduction in memory is achieved in all cases. For the PERK v2.2.0 parameter sets, increasing the time exponent by only 5 bits to match the KMP complexity yields memory reductions of 28, 41, and 49 bits, respectively, corresponding to an approximate 18% reduction relative to KMP. Analogous reductions for the remaining parameter sets are reported in Table 3.

All optimal parameter choices corresponding to Table 2 are reported in Table 4, and those corresponding to Table 3 are given in Table 5 in Appendix F.

Experimental Verification. To evaluate the effectiveness of our technique, we implemented a proof-of-concept attack based on Algorithm 1. For our experiments, we selected small parameters and instantiated the PKP problem with $n = 20$, $m = 12$, and $q = 47$. Using these parameters, we generated 1000 independent PKP instances and applied Algorithm 1 to each of them. The quantities reported below are averages over these 1000 trials.

For the algorithm, we set $k = 4$, $\ell = r = 6$, and $\mu_1 = \mu_2 = 2$. We generated the corresponding swap_sequences for ℓ and r via the SJT algorithm, together with the Compatibility Table, once at initialization and reused them across all instances.

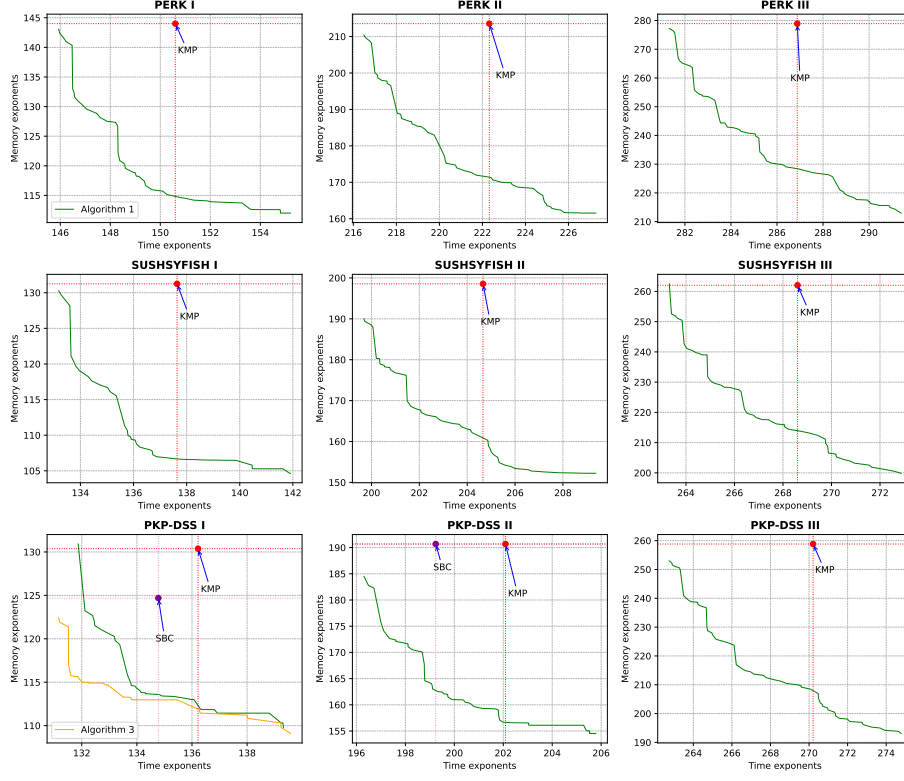


Fig. 5. Plot of memory vs time exponents trade-off of our algorithms compared with the KMP and SBC complexities.

The theoretical expected number of repetitions required until a randomly chosen pair of disjoint subsets (A_1, A_2) of $\text{Supp}(\mathbf{b})$ satisfies the conditions $A_1 \subseteq \text{Supp}(\mathbf{s}_1^*)$ and $A_2 \subseteq \text{Supp}(\mathbf{s}_2^*)$ is

$$\frac{\binom{n}{\mu_1} \binom{n-\mu_1}{\mu_2}}{\binom{\ell}{\mu_1} \binom{r}{\mu_2}} \approx 129.2.$$

In our experiments, the desired configuration occurred after an average of 129.68 repetitions over the 1000 runs, which closely matches the theoretical prediction. For efficiency, we continued with the rest of the procedure only once a valid pair of disjoint subsets (A_1, A_2) of $\text{Supp}(\mathbf{b})$ had been found.

To solve Eq. (5), we constructed the list $\mathcal{L}_1$ of pairs $(\mathbf{s}_1, \rho_\ell(\text{I-Supp}(\mathbf{s}_1)))$ organized by hash map using $\mathbf{M}_1 \mathbf{s}_1$, and enumerated the second list $\mathcal{L}_2$ of tuples $(\mathbf{t}_2 - \mathbf{M}_2 \mathbf{s}_2, \mathbf{s}_2, \rho_r(\text{I-Supp}(\mathbf{s}_2)))$. For the chosen parameters, each list has size

$$\binom{n - \mu_1 - \mu_2}{\ell - \mu_1} \ell! = 1,310,400 \quad \text{and} \quad \binom{n - \mu_1 - \mu_2}{r - \mu_2} r! = 1,310,400.$$

Performing a meet-in-the-middle search between these lists produced, on average, 351,891.66 collisions, which is consistent with the theoretical expectation

$$\frac{|\mathcal{L}_1| |\mathcal{L}_2|}{q^k} \approx 351,897.69.$$

After filtration using the Compatibility Table, the number of remaining vectors reduced to an average of 95,703.12, again matching the theoretical estimate

$$\frac{\binom{n-\mu_1-\mu_2}{\ell-\mu_1} \ell! \cdot \binom{n-\ell-\mu_2}{r-\mu_2} r!}{q^k} \approx 95,708.42.$$

From these remaining vectors, we successfully recovered the solution for every instance. Overall, the experimental results are consistent with the predicted behavior of Algorithm 1.

6.2 Results for Algorithm 2

We now evaluate the PERK v2.2.0, SUSHSYFISH and PKP-DSS parameter sets using Algorithm 2. Table 3 reports the time and memory bit-complexity exponents for KMP, for Algorithm 1 when its time complexity is increased to match that of KMP, and for Algorithm 2.

Table 3 indicates that, although Algorithm 2 does not achieve an exact square-root memory reduction as in the classical Schroeppel–Shamir framework due to the exponential growth of the permutation space, it nevertheless provides a substantial reduction in memory requirements across all parameter sets. Consequently, it provides improved overall efficiency compared to the existing approaches and offers lower memory usage than Algorithm 1 in memory-constrained regimes.

Figure 6 illustrates the trade-off between memory and time exponents across all parameter sets obtained using Algorithm 2. The results show that, for several parameter sets, a modest increase in the time exponent leads to a significant reduction in memory.

For PERK I, adjusting the parameters to $(k, k_1, \ell, \ell_1, r_1, \mu_1, \mu_2) = (11, 5, 23, 12, 12, 2, 7)$ raises the time from $2^{150.56}$ to $2^{150.98}$ while lowering the memory from $2^{97.94}$ to $2^{76.38}$. A similar effect appears for PERK III, where the choice $(23, 11, 40, 20, 20, 2, 7)$ slightly increases the time exponent to $2^{287.40}$ and simultaneously decreases the memory requirement from $2^{199.78}$ to $2^{144.36}$. For SUSHSYFISH I, the configuration $(11, 5, 22, 11, 11, 7, 2)$ shifts the time from $2^{137.61}$ to $2^{138.15}$ and reduces memory usage from $2^{79.98}$ to $2^{69.48}$. Finally, the configuration $(21, 10, 38, 19, 19, 7, 6)$ for SUSHSYFISH III yields a modest rise in time to $2^{269.38}$ together with a substantial drop in memory from $2^{207.62}$ to $2^{132.68}$.

Experimental Verifications. To validate the correctness of Algorithm 2, we conducted experiments on 1000 independent PKP instances with parameters $(n, m, q) = (20, 12, 47)$. As in the previous experiments, we set $k = 4$, $\ell = r = 6$ and $\mu_1 = \mu_2 = 2$ and for the new parameters introduced in Algorithm 2, we

Table 3. Time and Memory bit-complexity exponents for KMP, Algorithm 1 (when its time complexity is increased to match that of KMP), and our Schroeppe-Shamir based Algorithm 2 applied to the PERK v2.2.0, SUSHSYFISH and PKP-DSS parameter sets.

Schemes	n, m, q	KMP [20]		Algorithm 1		Algorithm 2	
		Time	Mem	Time	Mem	Time	Mem
PERK v2.2.0	64, 28, 2048	150.60	144.01	150.27	115.14	150.56	97.94
	92, 44, 2048	222.32	213.55	221.71	171.96	222.27	110.22
	118, 60, 2048	286.88	278.91	286.40	229.06	286.80	199.78
SUSHSYFISH	61, 29, 997	137.64	131.25	137.55	106.70	137.61	79.98
	87, 43, 1409	204.66	198.56	204.47	161.43	204.64	105.88
	111, 56, 1889	268.60	262.04	268.23	214.43	268.57	207.62
PKP-DSS	69, 42, 251	136.22	130.38	136.07	113.00	135.86	74.84
	94, 55, 509	202.09	190.68	202.01	156.68	201.75	99.58
	106, 48, 4093	270.21	258.83	270.09	208.44	268.82	134.78

used $k_1 = 1$, $k_2 = 3$, $\ell_1 = r_1 = 3$. For each instance, we first measured how many random trials were required to obtain a valid pair of disjoint subsets (A_1, A_2) of $\text{Supp}(\mathbf{b})$ of sizes μ_1 and μ_2 , respectively. For efficiency, once such a valid pair was found, we attempted to solve the valid instance by iterating over all $\mathbf{r} \in \mathbb{F}_q^{k_1}$ until a solution was recovered. The results reported below correspond to averages across all 1000 instances.

The expected number of trials needed for a randomly chosen disjoint pair (A_1, A_2) to satisfy the conditions $A_1 \subseteq \text{Supp}(\mathbf{s}_1^*)$ and $A_2 \subseteq \text{Supp}(\mathbf{s}_2^*)$ is

$$\frac{\binom{n}{\mu_1} \binom{n-\mu_1}{\mu_2}}{\binom{\ell}{\mu_1} \binom{r}{\mu_2}} \approx 129.2.$$

Across 1000 experiments, the observed average was 130.90 trials, which is in close agreement with the theoretical prediction. (This behaviour has also been experimentally confirmed earlier in the experimental verification of Algorithm 1.)

To generate the list $\mathcal{L}_1$, the algorithm iterates over all subsets $A_{11} \subseteq A_1$ satisfying

$$\max\{0, \mu_1 - \ell_2\} \leq |A_{11}| \leq \min\{\mu_1, \ell_1\},$$

and performs collision search and filtration over the lists $\mathcal{L}_{A_{11}}^{(1)}$ and $\mathcal{L}_{A_{12}}^{(1)}$. For a fixed $\mathbf{r} \in \mathbb{F}_q^{k_1}$, the expected total number of collisions across all such A_{11} is

$$\sum_{\max\{0, \mu_1 - \ell_2\} \leq i \leq \min\{\mu_1, \ell_1\}} \binom{\mu_1}{i} \frac{\binom{n-\mu_1-\mu_2}{\ell_1-i} \ell_1! \binom{n-\mu_1-\mu_2}{\ell_2-(\mu_1-i)} \ell_2!}{q^{k_1}} = 35,785.53,$$

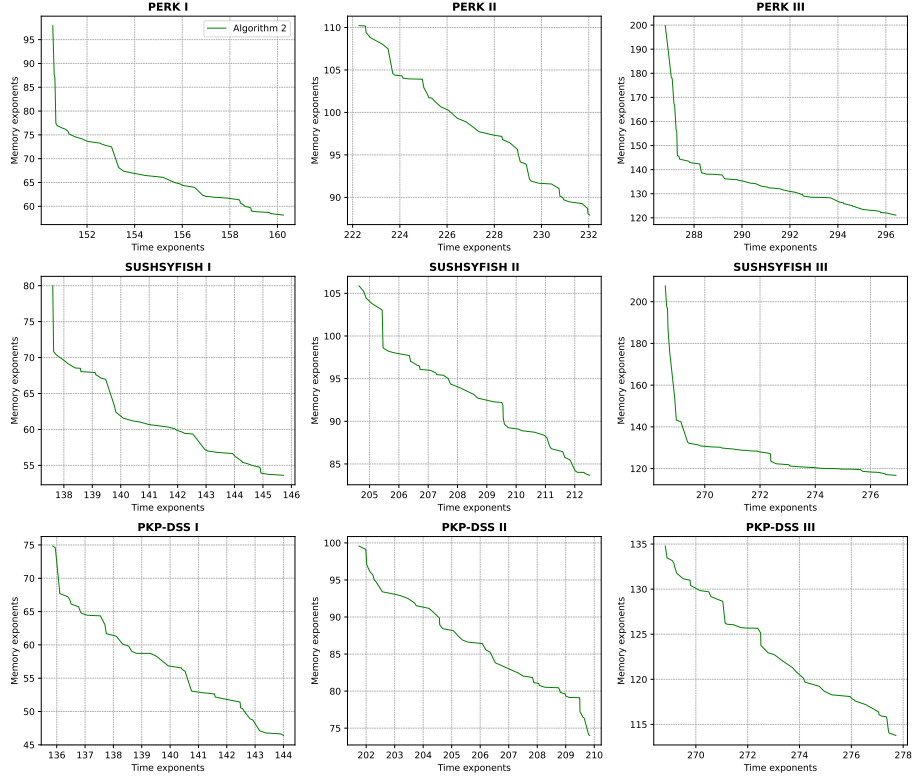


Fig. 6. Plot of memory vs time bit-complexity exponents trade-off for Algorithm 2.

and the expected size of $\mathcal{L}_1$ is

$$\frac{\binom{n-\mu_1-\mu_2}{\ell-\mu_1} \ell!}{q^{k_1}} = 27,880.85.$$

A similar calculation applies when generating $\mathcal{L}_2$, giving the same expected values of 35,785.53 collisions and 27,880.85 elements.

In our experiments, averaged over all $\mathbf{r} \in \mathbb{F}_q^{k_1}$ across all 1000 instances, we observed 35,785.30 collisions and a list size of 27,881.20 for $\mathcal{L}_1$, and 35,786.12 collisions and a list size of 27,881.01 for $\mathcal{L}_2$, all of which match the theoretical expectations closely.

Next, for a fixed $\mathbf{r} \in \mathbb{F}_q^{k_1}$, the expected number of collisions between the lists $\mathcal{L}_1$ and $\mathcal{L}_2$ is

$$\frac{|\mathcal{L}_1| |\mathcal{L}_2|}{q^{k_2}} = 7,487.18,$$

and the expected number of filtered solutions is

$$N_{\text{filtered}} = \frac{\binom{n-\mu_1-\mu_2}{\ell-\mu_1} \ell! \binom{n-\ell-\mu_2}{r-\mu_2} r!}{q^{k+k_1}} = 2,036.35.$$

Experimentally, averaged over all $\mathbf{r} \in \mathbb{F}_q^{k_1}$ across all 1000 instances, we observed 7,488.07 collisions and 2,036.85 filtered solutions, again in excellent agreement with the theoretical estimates.

Overall, these experiments confirm that Algorithm 2 behaves as predicted by our analysis. The empirical data closely matches the theoretical expectations at every stage, demonstrating the correctness of our Schroepel–Shamir adaptation for PKP.

7 Conclusion

In this work, we have presented a new constraint-based enumeration framework for solving the Permuted Kernel Problem by combining structured Steinhaus–Johnson–Trotter enumeration with Compatibility Table filtering. We have shown that our algorithm improves upon the KMP and SBC attacks. Notably, our cryptanalysis indicates recovery of the secret vector for all PERK v2.2.0 parameter sets below their estimated bit-security levels and yields improved complexity estimates for the SUSHSYFISH and PKP-DSS parameter sets. Moreover, we have introduced the Schroepel–Shamir style time–memory trade-off in the PKP setting, achieving substantial memory reductions while preserving essentially the same time complexity.

Overall, our results refine the current understanding of the concrete security of PKP-based signature schemes and show that structured enumeration techniques can lead to meaningful efficiency improvements in combinatorial cryptanalysis. These results also provide guidance for the selection of secure parameter sets in PKP-based constructions and motivate further investigation of related attack methodologies.

Acknowledgements. The authors would like to thank Andre Esser, Alessandro Budroni, Fukang Liu, and Floyd Zveydinger for valuable discussions.

References

1. Aaraj, N., Bettaieb, S., Bidoux, L., Budroni, A., Dyseryn, V., Esser, A., Feneuil, T., Gaborit, P., Kulkarni, M., Mateu, V., Palumbi, M., Perin, L., Rivain, M., Tillich, J.P., Xagawa, K.: PERK: Signature Scheme. Version 2.2.0, released December 2025 (2025), <https://pqc-perk.org/assets/downloads/perk-v2.2.0.pdf>
2. Baldi, M., Battagliola, M., Mechri, R.E., Santini, P., Schiavoni, R., Zuane, D.D.: SPECK: Signatures from permutation equivalence of codes and kernels. Cryptology ePrint Archive, Paper 2025/923 (2025), <https://eprint.iacr.org/2025/923>

3. Baritaud, T., Campana, M., Chauvaud, P., Gilbert, H.: On the security of the permuted kernel identification scheme. In: Brickell, E.F. (ed.) *Advances in Cryptology — CRYPTO’ 92*. pp. 305–311. Springer Berlin Heidelberg, Berlin, Heidelberg (1993). https://doi.org/10.1007/3-540-48071-4_21
4. Berger, T.P., Gueye, C.T., Klamti, J.B.: A np-complete problem in coding theory with application to code based cryptography. In: El Hajji, S., Nitaj, A., Souidi, E.M. (eds.) *Codes, Cryptology and Information Security*. pp. 230–237. Springer International Publishing, Cham (2017). https://doi.org/10.1007/978-3-319-55589-8_15
5. Bettaieb, S., Bidoux, L., Dyseryn, V., Esser, A., Gaborit, P., Kulkarni, M., Palumbi, M.: PERK: Compact Signature Scheme Based on a New Variant of the Permuted Kernel Problem. *Cryptology ePrint Archive*, Paper 2024/748 (2024). <https://doi.org/10.1007/s10623-024-01381-2>
6. Bettaieb, S., Bidoux, L., Gaborit, P., Kulkarni, M.: Modelings for generic PoK and applications: Shorter SD and PKP based signatures. *Cryptology ePrint Archive*, Paper 2024/1668 (2024), <https://eprint.iacr.org/2024/1668>
7. Beullens, W.: Not Enough LESS: An Improved Algorithm for Solving Code Equivalence Problems over $\mathbb{F}_q$. In: Dunkelman, O., Jr., M.J.J., O’Flynn, C. (eds.) *Selected Areas in Cryptography - SAC 2020 - 27th International Conference, Halifax, NS, Canada (Virtual Event), October 21-23, 2020, Revised Selected Papers*. Lecture Notes in Computer Science, vol. 12804, pp. 387–403. Springer (2020). https://doi.org/10.1007/978-3-030-81652-0_15
8. Beullens, W.: Sigma Protocols for MQ, PKP and SIS, and Fishy Signature Schemes. In: Canteaut, A., Ishai, Y. (eds.) *Advances in Cryptology – EUROCRYPT 2020*. pp. 183–211. Springer International Publishing, Cham (2020). https://doi.org/10.1007/978-3-030-45727-3_7
9. Beullens, W., Faugère, J.C., Koussa, E., Macario-Rat, G., Patarin, J., Perret, L.: PKP-based signature scheme. In: *Progress in Cryptology – INDOCRYPT 2019*. pp. 3–22. Springer International Publishing, Cham (2019). https://doi.org/10.1007/978-3-030-35423-7_1
10. Bidoux, L., Gaborit, P.: Compact post-quantum signatures from proofs of knowledge leveraging structure for the pkp, sd and rsd problems. In: El Hajji, S., Mesnager, S., Souidi, E.M. (eds.) *Codes, Cryptology and Information Security*. pp. 10–42. Springer Nature Switzerland, Cham (2023). https://doi.org/10.1007/978-3-031-33017-9_2
11. Budroni, A., Defranceschi, M., Pintore, F.: Recursion enabled: Improved cryptanalysis of the permuted kernel problem. *Cryptology ePrint Archive*, Paper 2025/2073 (2025), <https://eprint.iacr.org/2025/2073>, to appear at PQCrypto 2026
12. Ding, B., König, A.C.: Fast set intersection in memory. *Proc. VLDB Endow.* **4**(4), 255–266 (Jan 2011). <https://doi.org/10.14778/1938545.1938550>
13. Esser, A., Verbel, J.A., Zweydinger, F., Bellini, E.: Sok: Cryptographic estimators - a software library for cryptographic hardness estimation. In: *AsiaCCS*. ACM (2024). <https://doi.org/10.1145/3634737.3645007>
14. Georgiades, J.: Some remarks on the security of the identification scheme based on permuted kernels. *Journal of Cryptology* **5**, 133–137 (1992). <https://doi.org/10.1007/BF00193565>

15. Grover, L.K.: A fast quantum mechanical algorithm for database search. In: Proceedings of the Twenty-Eighth Annual ACM Symposium on Theory of Computing. p. 212–219. STOC '96, Association for Computing Machinery, New York, NY, USA (1996). <https://doi.org/10.1145/237814.237866>
16. Hartmanis, J.: Computers and Intractability: A Guide to the Theory of NP-Completeness (Michael R. Garey and David S. Johnson). SIAM Review **24**(1), 90–91 (1982). <https://doi.org/10.1137/1024022>
17. Jaulmes, É., Joux, A.: Cryptanalysis of pkp: A new approach. In: Kim, K. (ed.) Public Key Cryptography. pp. 165–172. Springer Berlin Heidelberg, Berlin, Heidelberg (2001). https://doi.org/10.1007/3-540-44586-2_12
18. Johnson, S.M.: Generation of permutations by adjacent transposition. Mathematics of Computation **17**(83), 282–285 (1963), <http://www.jstor.org/stable/2003846>
19. Knuth, D.: Art of Computer Programming, Volume 4A, The Combinatorial Algorithms, Part 1. Pearson Deutschland (2011), <https://elibrary.pearson.de/book/99.150005/9780133488876>
20. Koussa, E., Macario-Rat, G., Patarin, J.: On the complexity of the Permuted Kernel Problem. Cryptology ePrint Archive, Paper 2019/412 (2019), <https://eprint.iacr.org/2019/412>
21. LAMPE, R., PATARIN, J.: Analysis of some natural variants of the PKP algorithm. Cryptology ePrint Archive, Paper 2011/686 (2011), <https://eprint.iacr.org/2011/686>
22. Micciancio, D., Regev, O.: Lattice-based Cryptography, pp. 147–191. Springer Berlin Heidelberg (01 2009). https://doi.org/10.1007/978-3-540-88702-7_5
23. Paiva, T.B., Terada, R.: Cryptanalysis of the binary permuted kernel problem. Cryptology ePrint Archive, Paper 2021/398 (2021), <https://eprint.iacr.org/2021/398>
24. Patarin, J., Chauvaud, P.: Improved algorithms for the permuted kernel problem. In: Stinson, D.R. (ed.) Advances in Cryptology — CRYPTO' 93. pp. 391–402. Springer Berlin Heidelberg, Berlin, Heidelberg (1994). https://doi.org/10.1007/3-540-48329-2_33
25. Sanna, C.: On the number of solutions to a random instance of the permuted kernel problem. Journal of Complexity **86**, 101898 (2025). <https://doi.org/10.1016/j.jco.2024.101898>
26. Santini, P., Baldi, M., Chiaraluce, F.: A Novel Attack to the Permuted Kernel Problem. In: IEEE International Symposium on Information Theory, ISIT 2022, Espoo, Finland, June 26 - July 1, 2022. pp. 1441–1446. IEEE (2022). <https://doi.org/10.1109/ISIT50566.2022.9834867>
27. Santini, P., Baldi, M., Chiaraluce, F.: Computational Hardness of the Permuted Kernel and Subcode Equivalence Problems. Cryptology ePrint Archive, Paper 2022/1749 (2022). <https://doi.org/10.1109/TIT.2023.3323068>
28. Schroeppe, R., Shamir, A.: A $T = O(2^{n/2})$, $S = O(2^{n/4})$ Algorithm for Certain NP-Complete Problems. SIAM Journal on Computing **10**(3), 456–464 (1981), <https://doi.org/10.1137/0210033>
29. Shamir, A.: An Efficient Identification Scheme Based on Permuted Kernels. In: Brassard, G. (ed.) Advances in Cryptology — CRYPTO' 89 Proceedings. pp. 606–609. Springer New York, New York, NY (1990). https://doi.org/10.1007/0-387-34805-0_54

Appendix

A Example for index-support (I-Supp) set

Consider the vector $\mathbf{b} = (3, 2, 7, 9, 4, 6)^T \in \mathbb{F}_{17}^6$, with parameters $\ell = 2$ and $r = 2$. Let $\mathbf{s}_1 = (4, 2)^T$ and $\mathbf{s}_2 = (6, 3)^T$. Clearly, $\mathbf{s}_1$ and $\mathbf{s}_2$ have no elements in common, so they are compatible. The corresponding index-support sets (I-Supp) are:

$$\text{I-Supp}(\mathbf{s}_1) = \{1, 4\}, \quad \text{I-Supp}(\mathbf{s}_2) = \{0, 5\}.$$

As $\text{I-Supp}(\mathbf{s}_1) \cap \text{I-Supp}(\mathbf{s}_2) = \emptyset$, this confirms that $\mathbf{s}_1$ and $\mathbf{s}_2$ are indeed compatible.

B A proof of Correctness of the Rank Function

Let $S = \{p_0, p_1, \dots, p_{t-1}\} \subseteq [n]$ with $p_0 < p_1 < \dots < p_{t-1}$. Under the relation $<$, the rank $\rho_t(S)$ is defined to be the number of t -subsets $S' \subseteq [n]$ that precede S in lexicographic order. Any such subset $S' = \{p'_0, \dots, p'_{t-1}\}$ differs from S for the first time at a unique index $j \in \{0, \dots, t-1\}$, meaning that $p'_\delta = p_\delta$ for all $\delta < j$ and $p'_j < p_j$.

Fix such an index j . Since the elements of S' must be strictly increasing, the possible values of p'_j are precisely the integers i with $p_{j-1} < i < p_j$, where by convention $p_{-1} = -1$. Once $p'_j = i$ is chosen, the remaining $t-j-1$ elements of S' must be selected from the set $\{i+1, i+2, \dots, n-1\}$, which has cardinality $n-i-1$. The number of ways to complete S' in this case is therefore $\binom{n-i-1}{t-j-1}$.

Thus, for a fixed j , the number of t -subsets that agree with S up to position $j-1$ and are lexicographically smaller than S at position j is

$$\sum_{i=p_{j-1}+1}^{p_j-1} \binom{n-i-1}{t-j-1} = \binom{n-p_{j-1}-1}{t-j} - \binom{n-p_j}{t-j}.$$

As the index j ranges from 0 to $t-1$, these sets of subsets are disjoint and together exhaust all t -subsets that precede S in lexicographic order. Summing over all j therefore gives

$$\rho_t(S) = \sum_{j=0}^{t-1} \left[\binom{n-p_{j-1}-1}{t-j} - \binom{n-p_j}{t-j} \right],$$

which is exactly the number of t -subsets of $[n]$ that are lexicographically smaller than S . Consequently, ρ_t assigns to each t -subset a unique integer in $\left[\binom{n}{t} \right]$, and hence defines a valid rank function for the lexicographic ordering.

C Example for Compatibility Table

Let $n = 5$ and $\ell = 2$, and consider the ℓ -subset $S = \{2, 4\} \subseteq [5]$. Here, $p_0 = 2$ and $p_1 = 4$, with $p_{-1} = -1$. Then,

$$\rho_2(S) = \sum_{j=0}^{2-1} \left[\binom{5-p_{j-1}-1}{2-j} - \binom{5-p_j}{2-j} \right] = \binom{5}{2} - \binom{3}{2} + \binom{2}{1} - \binom{1}{1} = 10 - 3 + 2 - 1 = 8.$$

A complete enumeration of all $\binom{5}{2} = 10$ subsets of size 2 and their ranks under ρ_2 are

$$\begin{aligned} \rho_2(\{0, 1\}) &= 0, \rho_2(\{0, 2\}) = 1, \rho_2(\{0, 3\}) = 2, \rho_2(\{0, 4\}) = 3, \rho_2(\{1, 2\}) = 4, \\ \rho_2(\{1, 3\}) &= 5, \rho_2(\{1, 4\}) = 6, \rho_2(\{2, 3\}) = 7, \rho_2(\{2, 4\}) = 8, \rho_2(\{3, 4\}) = 9. \end{aligned}$$

Let us construct the Compatibility Table $\mathcal{A}$. Since $\ell = r = 2$, the array $\mathcal{A}$ is a 10×10 matrix. Its (i, j) -entry is 1 if and only if the two subsets indexed by i and j are disjoint. For instance:

- The subsets $\{0, 1\}$ and $\{2, 4\}$ are disjoint, and their ranks under ρ_2 are 0 and 8, respectively. Hence, $\mathcal{A}[0][8] = 1$.
- The subsets $\{2, 4\}$ and $\{2, 3\}$ intersect in the element 2, and the rank of $\{2, 3\}$ under ρ_2 is 7. Therefore, $\mathcal{A}[8][7] = 0$.

Carrying out this procedure for all pairs, we obtain the full Compatibility Table:

$$\mathcal{A} = \begin{pmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \end{pmatrix}$$

D Asymptotic Analysis of Algorithm 1

In this section, we analyze the asymptotic complexity of Algorithm 1. We begin by recalling several standard approximations. For two functions $f_1(n)$ and $f_2(n)$, we write $f_1(n) \sim f_2(n)$ if

$$\lim_{n \rightarrow \infty} \frac{f_1(n)}{f_2(n)} = 1.$$

A fundamental approximation is Stirling's formula, which states that

$$n! \sim \frac{1}{\sqrt{2\pi n}} \left(\frac{n}{e} \right)^n = 2^{n \log_2 \left(\frac{n}{e} \right) (1+o(1))}.$$

For $x \in [0, 1]$, the binary entropy function is defined by

$$H_2(x) = -x \log_2(x) - (1-x) \log_2(1-x),$$

and satisfies the approximation

$$\binom{n}{\mu n} \sim 2^{nH_2(\mu)(1+o(1))}.$$

We express q as a function of the code rate $R = \frac{n-m}{n}$. We assume that q is the smallest integer such that a random PKP instance has at most one solution on average, that is, $\frac{n!}{q^m} \leq 1$. Using Stirling's approximation, this yields

$$q \sim \left(\frac{n}{e}\right)^{\frac{1}{1-R}} (1 + o(1)).$$

We now show that Algorithm 1 does not provide an asymptotic improvement over the KMP algorithm. The asymptotic complexity of KMP is given by

$$T_{\text{KMP}} = \max \left\{ {}^nP_\ell, {}^nP_r, \frac{{}^nP_\ell {}^nP_r}{q^k} \right\}.$$

For optimization, setting $\ell = r = \tau n$ with $R/2 < \tau < 1/2$, Santini et al. [27] showed that

$$T_{\text{KMP}} = 2^{nc_{\text{KMP}}^*},$$

where

$$c_{\text{KMP}}^* = H_2(\tau^*) + \tau^* \log \left(\frac{n\tau^*}{e} \right), \quad \text{and} \quad \tau^* = \frac{R}{1+R}.$$

We now derive the asymptotic complexity of Algorithm 1 following a similar approach. Focusing on the dominant term, we obtain

$$T_{\text{Alg-1}} = \frac{\binom{n}{\mu_1} \binom{n-\mu_1}{\mu_2}}{\binom{\ell}{\mu_1} \binom{r}{\mu_2}} \max \left\{ \binom{n-\mu_1-\mu_2}{\ell-\mu_1} \ell!, \binom{n-\mu_1-\mu_2}{r-\mu_2} r!, \frac{\binom{n-\mu_1-\mu_2}{\ell-\mu_1} \ell! \binom{n-\mu_1-\mu_2}{r-\mu_2} r!}{q^k} \right\}.$$

To optimize, we balance the three terms by setting $\ell = r = \tau n$ with $R/2 < \tau < 1/2$, and $\mu_1 = \mu_2 = \gamma n$ with $0 \leq \gamma < \tau$. Equality of the dominant terms yields

$$\binom{n-2\gamma n}{\tau n - \gamma n} (\tau n)! = \frac{\binom{n-2\gamma n}{\tau n - \gamma n}^2 ((\tau n)!)^2}{q^{2\tau n - Rn}}.$$

Applying the asymptotic approximations, we obtain

$$\frac{n(2\tau - R)}{1 - R} \log \left(\frac{n}{e} \right) = n(1 - 2\gamma) H_2 \left(\frac{\tau - \gamma}{1 - 2\gamma} \right) + \tau n \log \left(\frac{\tau n}{e} \right).$$

Dividing by $n \log n$ and letting $n \rightarrow \infty$, we obtain $\frac{2\tau-R}{1-R} = \tau$. Solving this equation gives $\tau^* = \frac{R}{1+R}$ as the optimal parameter.

Substituting this into the complexity expression, we obtain

$$T_{\text{Alg-1}} = 2^{nc_{\text{Alg-1}}^*}, \quad \text{where}$$

$$\begin{aligned} c_{\text{Alg-1}}^* &= H_2(\gamma) + (1-\gamma)H_2\left(\frac{\gamma}{1-\gamma}\right) - 2\tau^*H_2\left(\frac{\gamma}{\tau^*}\right) \\ &\quad + (1-2\gamma)H_2\left(\frac{\tau^*-\gamma}{1-2\gamma}\right) + \tau^* \log\left(\frac{n\tau^*}{e}\right). \end{aligned}$$

To optimize over γ , define

$$\begin{aligned} f(\gamma) &:= H_2(\gamma) + (1-\gamma)H_2\left(\frac{\gamma}{1-\gamma}\right) - 2\tau^*H_2\left(\frac{\gamma}{\tau^*}\right) + (1-2\gamma)H_2\left(\frac{\tau^*-\gamma}{1-2\gamma}\right) \\ &= -2\tau^* \log(\tau^*) + (\tau^* - \gamma) \log(\tau^* - \gamma) - (1 - \tau^* - \gamma) \log(1 - \tau^* - \gamma). \end{aligned}$$

We choose γ^* such that $f(\gamma^*)$ becomes minimum. Differentiating f , we obtain

$$f'(\gamma) = \log\left(\frac{1 - \tau^* - \gamma}{\tau^* - \gamma}\right).$$

Since $\tau^* < \frac{1}{2}$ and $\gamma < \tau^*$, we have $f'(\gamma) > 0$ for all $\gamma \in [0, \tau^*)$. Hence, f is strictly increasing and attains its minimum at $\gamma^* = 0$.

Substituting $\gamma^* = 0$, we obtain

$$c_{\text{Alg-1}}^* = H_2(\tau^*) + \tau^* \log\left(\frac{n\tau^*}{e}\right), \quad \text{where, } \tau^* = \frac{R}{1+R},$$

which coincides with c_{KMP}^* . Therefore, Algorithm 1 does not provide an asymptotic improvement over the KMP algorithm.

E Pseudocode for the Time-Memory trade-off

Algorithm 2: A Memory-Efficient Schroeppe-Shamir Framework for PKP

Input: $(\tilde{\mathbf{A}}, \mathbf{b}) \in \mathbb{F}_q^{m \times n} \times \mathbb{F}_q^n$

- 1 Choose optimal k, k_1, ℓ such that $1 \leq k \leq m, 0 \leq \ell \leq n - m + k, 0 \leq k_1 \leq k$, and compute $w = n - m + k, r = w - \ell, k_2 = k - k_1$;
- 2 Choose optimal $\ell_1, r_1, \mu_1, \mu_2$, such that $0 \leq \ell_1 \leq \ell, 0 \leq r_1 \leq r, 0 \leq \mu_1 \leq \ell, 0 \leq \mu_2 \leq r$, and compute $\ell_2 = \ell - \ell_1, r_2 = r - r_1$;
- 3 Find an invertible matrix $\mathbf{Q}$ such that $\mathbf{Q}\tilde{\mathbf{A}}$ matches the structure in Fig. 3 and compute $\mathbf{t} = \mathbf{Q}(\sigma, 0, \dots, 0)^T$;
- 4 Set $\mathbf{t}_1 \leftarrow \mathbf{t}[m - k :]$, $\mathbf{t}_2 \leftarrow \mathbf{t}[m - k :]$, $\mathbf{t}_2^{(2)} \leftarrow \mathbf{t}_2[k_2 :]$ and $\mathbf{t}_2^{(1)} \leftarrow \mathbf{t}_2[k_2 :]$
- 5 **repeat**
- 6 Choose disjoint random subsets $\Lambda_1, \Lambda_2 \subseteq \text{Supp}(\mathbf{b})$ such that $|\Lambda_1| = \mu_1$ and $|\Lambda_2| = \mu_2$;
- 7 Compute $\Lambda \leftarrow \Lambda_1 \cup \Lambda_2$;
- 8 **for all** $\mathbf{r} \in \mathbb{F}_q^{k_1}$ **do**
- 9 // Stage 1: Search for collisions in the lower k_1 coordinates using the Schroeppe-Shamir split
- 10 $\mathcal{L}_1 \leftarrow \emptyset$;
- 11 **for all** $\Lambda_{11} \subseteq \Lambda_1$ such that $|\Lambda_{11}| \leq \ell_1$ and $\mu_1 - |\Lambda_{11}| \leq \ell_2$ **do**
- 12 $\mathcal{L}_{\Lambda_{11}}^{(1)} \leftarrow \emptyset, \Lambda_{12} \leftarrow \Lambda_1 \setminus \Lambda_{11}$;
- 13 // Lists $\mathcal{L}_{\Lambda_{11}}^{(1)}$ and $\mathcal{L}_{\Lambda_{12}}^{(1)}$ are generated via the SJT algorithm with incremental updates (Section 4.1), using hash maps.
- 14 $\mathcal{L}_{\Lambda_{11}}^{(1)} \leftarrow \left\{ \left(\mathbf{M}_{11}^{(1)} \mathbf{s}_{11}, \mathbf{s}_{11} \right) \mid \mathbf{s}_{11} \in \mathcal{S}_{\ell_1}(\mathbf{b}), \Lambda_{11} \subseteq \text{Supp}(\mathbf{s}_{11}), \text{Supp}(\mathbf{s}_{11}) \setminus \Lambda_{11} \subseteq \text{Supp}(\mathbf{b}) \setminus \Lambda \right\}$;
- 15 $\mathcal{L}_{\Lambda_{12}}^{(1)} \leftarrow \left\{ \left(\mathbf{r} - \mathbf{M}_{12}^{(1)} \mathbf{s}_{12}, \mathbf{s}_{12} \right) \mid \mathbf{s}_{12} \in \mathcal{S}_{\ell_2}(\mathbf{b}), \Lambda_{12} \subseteq \text{Supp}(\mathbf{s}_{12}), \text{Supp}(\mathbf{s}_{12}) \setminus \Lambda_{12} \subseteq \text{Supp}(\mathbf{b}) \setminus \Lambda \right\}$;
- 16 **for** $(\mathbf{c}, \mathbf{s}_{11}) \in \mathcal{L}_{\Lambda_{11}}^{(1)}$ and $(\mathbf{d}, \mathbf{s}_{12}) \in \mathcal{L}_{\Lambda_{12}}^{(1)}$ **do**
- 17 **if** $\mathbf{c} = \mathbf{d}$ and $(\mathbf{s}_{11} \parallel \mathbf{s}_{12}) \in \mathcal{S}_\ell(\mathbf{b})$ **then**
- 18 $\mathcal{L}_{\Lambda_{11}} \leftarrow \mathcal{L}_{\Lambda_{11}} \cup \left\{ \left(\mathbf{M}_1^{(2)}(\mathbf{s}_{11} \parallel \mathbf{s}_{12}), (\mathbf{s}_{11} \parallel \mathbf{s}_{12}) \right) \right\}$
- 19 $\mathcal{L}_1 \leftarrow \mathcal{L}_1 \cup \mathcal{L}_{\Lambda_{11}}$;
- 20 $\mathcal{L}_2 \leftarrow \emptyset$;
- 21 **for all** $\Lambda_{21} \subseteq \Lambda_2$ such that $|\Lambda_{21}| \leq r_1$ and $\mu_2 - |\Lambda_{21}| \leq r_2$ **do**
- 22 $\mathcal{L}_{\Lambda_{21}} \leftarrow \emptyset, \Lambda_{22} \leftarrow \Lambda_2 \setminus \Lambda_{21}$;
- 23 // Lists $\mathcal{L}_{\Lambda_{21}}^{(1)}$ and $\mathcal{L}_{\Lambda_{22}}^{(1)}$ are generated via the SJT algorithm with incremental updates (Section 4.1), using hash maps.
- 24 $\mathcal{L}_{\Lambda_{21}}^{(1)} \leftarrow \left\{ \left(\mathbf{M}_{21}^{(1)} \mathbf{s}_{21}, \mathbf{s}_{21} \right) \mid \mathbf{s}_{21} \in \mathcal{S}_{r_1}(\mathbf{b}), \Lambda_{21} \subseteq \text{Supp}(\mathbf{s}_{21}), \text{Supp}(\mathbf{s}_{21}) \setminus \Lambda_{21} \subseteq \text{Supp}(\mathbf{b}) \setminus \Lambda \right\}$;
- 25 $\mathcal{L}_{\Lambda_{22}}^{(1)} \leftarrow \left\{ \left(\mathbf{t}_2^{(1)} - \mathbf{r} - \mathbf{M}_{22}^{(1)} \mathbf{s}_{22}, \mathbf{s}_{22} \right) \mid \mathbf{s}_{22} \in \mathcal{S}_{r_2}(\mathbf{b}), \Lambda_{22} \subseteq \text{Supp}(\mathbf{s}_{22}), \text{Supp}(\mathbf{s}_{22}) \setminus \Lambda_{22} \subseteq \text{Supp}(\mathbf{b}) \setminus \Lambda \right\}$;
- 26 **for** $(\mathbf{c}, \mathbf{s}_{21}) \in \mathcal{L}_{\Lambda_{21}}^{(1)}$ and $(\mathbf{d}, \mathbf{s}_{22}) \in \mathcal{L}_{\Lambda_{22}}^{(1)}$ **do**
- 27 **if** $\mathbf{c} = \mathbf{d}$ and $(\mathbf{s}_{21} \parallel \mathbf{s}_{22}) \in \mathcal{S}_r(\mathbf{b})$ **then**
- 28 $\mathcal{L}_{\Lambda_{21}} \leftarrow \mathcal{L}_{\Lambda_{21}} \cup \left\{ \left(\mathbf{t}_2^{(2)} - \mathbf{M}_2^{(2)}(\mathbf{s}_{21} \parallel \mathbf{s}_{22}), (\mathbf{s}_{21} \parallel \mathbf{s}_{22}) \right) \right\}$
- 29 $\mathcal{L}_2 \leftarrow \mathcal{L}_2 \cup \mathcal{L}_{\Lambda_{21}}$;
- 30 // Stage 2: Merge $\mathcal{L}_1$ and $\mathcal{L}_2$ by matching upper k_2 coordinates
- 31 **for** $(\mathbf{c}, \mathbf{s}_1) \in \mathcal{L}_1$ and $(\mathbf{d}, \mathbf{s}_2) \in \mathcal{L}_2$ **do**
- 32 **if** $\mathbf{c} = \mathbf{d}$ and $(\mathbf{s}_1 \parallel \mathbf{s}_2) \in \mathcal{S}_w(\mathbf{b})$ **then**
- 33 $\mathbf{x}_2 \leftarrow (\mathbf{s}_1 \parallel \mathbf{s}_2)$;
- 34 compute $\mathbf{x}_1 \leftarrow \mathbf{t}_1 - \mathbf{M}' \mathbf{x}_2$;
- 35 **if** $(\mathbf{x}_1 \parallel \mathbf{x}_2)$ forms a permutation of $\mathbf{b}$ **then**
- 36 **return** $(\mathbf{x}_1 \parallel \mathbf{x}_2)$;
- 37 **until** Solution found;

F Optimal parameter choices associated with the Tables

Table 4. Optimal parameter choices associated with Table 2

Schemes	n, m, q	KMP [20] (k, ℓ)	SBC [27] (d, w, w_1, k)	Algorithm 1 (k, ℓ, μ_1, μ_2)	Algorithm 3 ($d, w, w_1, k, \mu_1, \mu_2$)
PERK v2.2.0	64, 28, 2048	(12, 24)	(12, 48, 24, 13)	(12, 24, 1, 0)	(12, 48, 24, 14, 1, 0)
	92, 44, 2048	(18, 33)	(18, 66, 33, 19)	(18, 33, 2, 1)	(18, 66, 33, 20, 3, 0)
	118, 60, 2048	(24, 41)	(24, 82, 41, 25)	(24, 41, 1, 1)	(24, 82, 41, 26, 1, 0)
SUSHSYFISH	61, 29, 997	(12, 22)	(12, 44, 22, 13)	(12, 22, 1, 0)	(12, 44, 22, 14, 1, 0)
	87, 43, 1409	(18, 31)	(18, 62, 31, 19)	(17, 30, 0, 5)	(18, 62, 31, 20, 0, 0)
	111, 56, 1889	(23, 39)	(23, 78, 39, 24)	(23, 39, 0, 0)	(23, 78, 39, 25, 0, 0)
PKP-DSS	69, 42, 251	(15, 21)	(1, 22, 2, 15)	(15, 21, 0, 0)	(1, 21, 2, 15, 0, 1)
	94, 55, 509	(20, 29)	(1, 31, 2, 21)	(19, 29, 3, 3)	(1, 31, 5, 19, 11, 0)
	106, 48, 4093	(21, 39)	(21, 79, 39, 22)	(20, 39, 3, 3)	(20, 78, 39, 22, 5, 0)

Table 5. Optimal parameter choices associated with Table 3

Schemes	n, m, q	Algorithm 1 (k, ℓ, μ_1, μ_2)	Algorithm 2 ($k, k_1, \ell, \ell_1, r_1, \mu_1, \mu_2$)
PERK v2.2.0	64, 28, 2048	(10, 21, 1, 18)	(12, 4, 24, 8, 8, 1, 1)
	92, 44, 2048	(15, 29, 2, 24)	(17, 8, 32, 16, 16, 3, 8)
	118, 60, 2048	(20, 36, 1, 28)	(24, 7, 41, 12, 12, 1, 1)
SUSHSYFISH	61, 29, 997	(10, 19, 1, 16)	(12, 5, 22, 10, 9, 1, 0)
	87, 43, 1409	(15, 27, 0, 22)	(17, 8, 30, 14, 15, 0, 5)
	111, 56, 1889	(19, 34, 0, 27)	(23, 5, 39, 9, 9, 0, 0)
PKP-DSS	69, 42, 251	(13, 18, 0, 13)	(14, 6, 20, 9, 10, 0, 4)
	94, 55, 509	(16, 25, 2, 20)	(19, 9, 29, 14, 14, 5, 3)
	106, 48, 4093	(17, 35, 5, 29)	(20, 10, 39, 19, 19, 2, 2)

G The SBC Algorithm

Assume that ISD techniques applied to Eq. (1) produce a linear system of d equations in w variables:

$$\tilde{\mathbf{H}}\tilde{\mathbf{x}} = \tilde{\mathbf{t}}, \quad (6)$$

where, $\tilde{\mathbf{H}} \in \mathbb{F}_q^{d \times w}$, $\tilde{\mathbf{x}} \in \mathbb{F}_q^w$, $\tilde{\mathbf{t}} \in \mathbb{F}_q^d$ and; both d and w are optimization parameters.

Analogously to the KMP approach, the vectors $\tilde{\mathbf{x}}$ and $\tilde{\mathbf{H}}$ are partitioned as $\tilde{\mathbf{x}} = (\tilde{\mathbf{x}}_1 \parallel \tilde{\mathbf{x}}_2)$ and $\tilde{\mathbf{H}} = (\tilde{\mathbf{H}}_1 \parallel \tilde{\mathbf{H}}_2)$, where $\tilde{\mathbf{x}}_1 \in \mathbb{F}_q^{w_1}$, $\tilde{\mathbf{x}}_2 \in \mathbb{F}_q^{w_2}$ with $w_2 = w - w_1$, and $\tilde{\mathbf{H}}_1, \tilde{\mathbf{H}}_2$ are of appropriate dimensions. Here, w_1 is treated as an optimization parameter. Substituting these into Eq. (6) yields:

$$\tilde{\mathbf{H}}_1 \tilde{\mathbf{x}}_1 = \tilde{\mathbf{t}} - \tilde{\mathbf{H}}_2 \tilde{\mathbf{x}}_2. \quad (7)$$

To proceed, two sets are defined as follows:

$$\begin{aligned} \mathcal{K}_1 &= \left\{ \left(\tilde{\mathbf{H}}_1 \tilde{\mathbf{x}}_1, \tilde{\mathbf{x}}_1 \right) \mid \tilde{\mathbf{x}}_1 \in \mathcal{S}_{w_1}(\mathbf{b}) \right\}, \\ \mathcal{K}_2 &= \left\{ \left(\tilde{\mathbf{t}} - \tilde{\mathbf{H}}_2 \tilde{\mathbf{x}}_2, \tilde{\mathbf{x}}_2 \right) \mid \tilde{\mathbf{x}}_2 \in \mathcal{S}_{w_2}(\mathbf{b}) \right\}. \end{aligned}$$

A meet-in-the-middle approach, followed by a compatibility filtration, is then used to find collisions between $\mathcal{K}_1$ and $\mathcal{K}_2$ based on Eq. (7). The compatible matching pairs $(\tilde{\mathbf{x}}_1, \tilde{\mathbf{x}}_2)$ yield candidates $\tilde{\mathbf{x}} = (\tilde{\mathbf{x}}_1 \parallel \tilde{\mathbf{x}}_2)$ satisfying Eq. (6) and thus we recover the set:

$$\mathcal{K} = \left\{ \tilde{\mathbf{x}} \in \mathcal{S}_w(\mathbf{b}) \mid \tilde{\mathbf{H}}\tilde{\mathbf{x}} = \tilde{\mathbf{t}} \right\}.$$

To reintegrate the solution candidates into the original system, a permutation matrix $\mathbf{S} \in \mathbb{F}_q^{n \times n}$ is introduced, such that:

$$\mathbf{S}\mathbf{x}^* = (\hat{\mathbf{x}}^* \parallel \tilde{\mathbf{x}}^*), \quad (8)$$

i.e., the w coordinates corresponding to variables in Eq. (6) are moved to the last w positions of $\mathbf{x}^*$. Applying this permutation to Eq. (1) gives:

$$(\tilde{\mathbf{A}}\mathbf{S}^{-1})(\hat{\mathbf{x}}^* \parallel \tilde{\mathbf{x}}^*) = (\sigma, 0, \dots, 0)^T. \quad (9)$$

An invertible matrix $\tilde{\mathbf{Q}}$ is then applied to both sides of Eq. (9) in order to bring $\tilde{\mathbf{A}}\mathbf{S}^{-1}$ into a semi-systematic form:

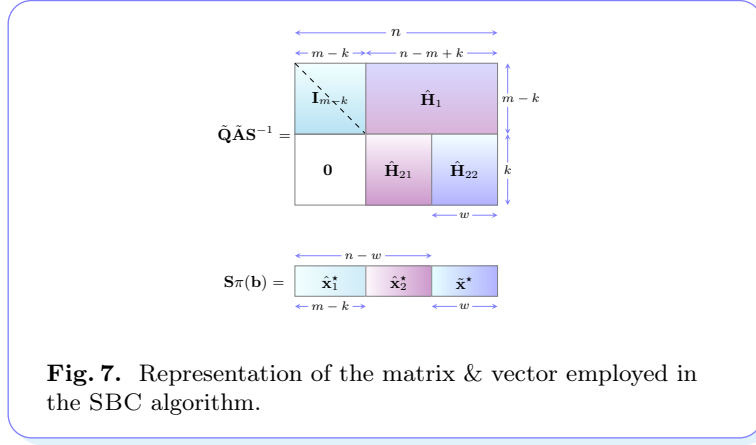
$$\tilde{\mathbf{Q}}\tilde{\mathbf{A}}\mathbf{S}^{-1} = \begin{pmatrix} \mathbf{I}_{m-k} & \hat{\mathbf{H}}_1 \\ \mathbf{0} & \hat{\mathbf{H}}_2 \end{pmatrix},$$

where, $\hat{\mathbf{H}}_1, \hat{\mathbf{H}}_2$ are of appropriate dimensions, $\hat{\mathbf{t}} = \tilde{\mathbf{Q}}(\sigma, 0, \dots, 0)^T$, and k is an optimization parameter. Letting $\hat{\mathbf{x}} = (\hat{\mathbf{x}}_1 \parallel \hat{\mathbf{x}}_2)$, with dimensions $m - k$ and $n - w - m + k$ respectively, the system becomes:

$$\hat{\mathbf{x}}_1 = \hat{\mathbf{t}}_1 - \hat{\mathbf{H}}_1(\hat{\mathbf{x}}_2 \parallel \tilde{\mathbf{x}}), \quad (10)$$

$$\hat{\mathbf{H}}_{21}\hat{\mathbf{x}}_2 = \hat{\mathbf{t}}_2 - \hat{\mathbf{H}}_{22}\tilde{\mathbf{x}}, \quad (11)$$

where $\hat{\mathbf{H}}_2 = (\hat{\mathbf{H}}_{21} \parallel \hat{\mathbf{H}}_{22})$, and $\hat{\mathbf{t}} = (\hat{\mathbf{t}}_1 \parallel \hat{\mathbf{t}}_2)$, with appropriate dimensions. To visualize the matrices and the vectors which are used in this algorithm, we refer to Figure 7.



Note that Eq. (11) contains k equations; however, since d equations have already been accounted for in Eq. (6), the effective number of linearly independent equations in Eq. (11) is reduced to $k - d$. Consequently, it suffices to work with only the first $k - d$ equations.

Now further two lists are generated:

$$\begin{aligned}\mathcal{L}_1 &= \left\{ \left(\hat{\mathbf{H}}_{21} \hat{\mathbf{x}}_2, \hat{\mathbf{x}}_2 \right) \mid \hat{\mathbf{x}}_2 \in \mathcal{S}_{n-w-m+k}(\mathbf{b}) \right\}, \\ \mathcal{L}_2 &= \left\{ \left(\hat{\mathbf{t}}_2 - \hat{\mathbf{H}}_{22} \tilde{\mathbf{x}}, \tilde{\mathbf{x}} \right) \mid \tilde{\mathbf{x}} \in \mathcal{K} \right\}.\end{aligned}$$

Again applying the meet-in-the-middle strategy, followed by a compatibility filtration, based on Eq. (11), the following set is generated

$$\mathcal{L} = \left\{ (\hat{\mathbf{x}}_2 \parallel \tilde{\mathbf{x}}) \in \mathcal{S}_{n-m+k}(\mathbf{b}) \mid \hat{\mathbf{H}}_{21} \hat{\mathbf{x}}_2 = \hat{\mathbf{t}}_2 - \hat{\mathbf{H}}_{22} \tilde{\mathbf{x}} \right\}.$$

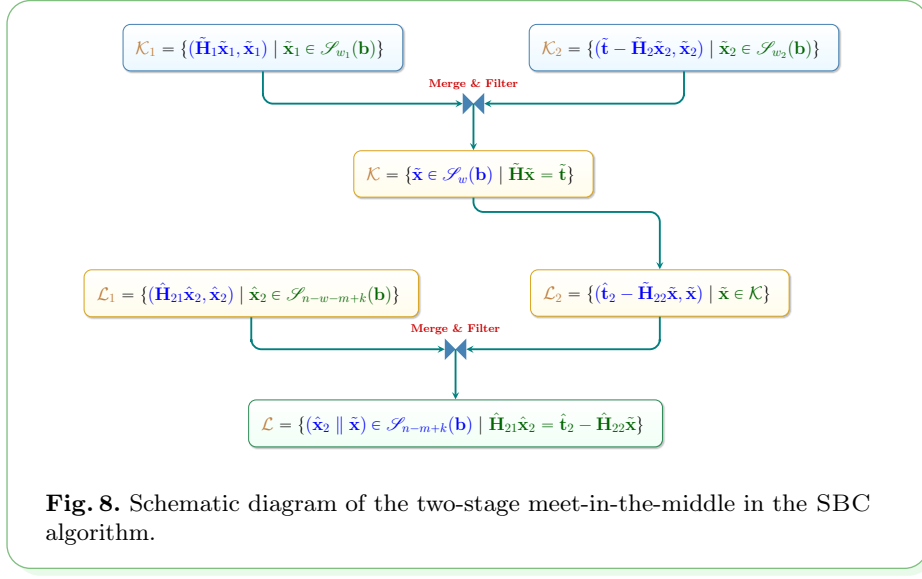
Figure 8 illustrates this two-stage meet-in-the-middle attack.

For each candidate in $\mathcal{L}$, the vector $\hat{\mathbf{x}}_1$ is computed using Eq. (10), after which the full vector $\mathbf{S}\mathbf{x}$ is reconstructed. A final check is then performed to determine whether $\mathbf{S}\mathbf{x} \in \mathcal{S}_n(\mathbf{b})$; if so, the corresponding vector $\mathbf{x}$ is output as a valid solution to the PKP instance.

We now analyze the complexity of the SBC algorithm.

Complexity Analysis. The complexity of the SBC algorithm mainly consists of two components: the cost of identifying d equations supported on w variables using ISD techniques, and the cost of generating the lists $\mathcal{K}$ and $\mathcal{L}$. As analyzed in [26], the first part is evaluated by the term

$$T_{\text{ISD}}^{(d)}(n, m, w) = \mathcal{O}\left(\frac{m^3 + \binom{m}{d}}{p^{(d)}(n, m, w)}\right),$$



where

$$p^{(d)}(n, m, w) = \min \left\{ \frac{\binom{w}{d} \binom{n-w}{m-d}}{\binom{n}{m}} \tilde{N}_{w,d}, 1 \right\}, \quad \tilde{N}_{w,d} = \binom{n}{w} (q^d - 1)^{w-d} \frac{\prod_{i=0}^{d-1} \frac{1 - q^{m-i}}{1 - q^{i+1}}}{\prod_{i=0}^{d-1} \frac{1 - q^{n-i}}{1 - q^{i+1}}} \quad \text{and,} \quad \tilde{N}_{w,d} \geq 1.$$

Here, $\tilde{N}_{w,d}$ denotes a lower bound of the expected number of subcodes of a randomly chosen code $\mathcal{C} \subseteq \mathbb{F}_q^n$ of dimension m , having dimension d and support size w . See [26] for further details.

To evaluate the remaining part of the complexity, we first consider Eq. (6). Since $|\mathcal{S}_{w_1}(\mathbf{b})| = {}^n P_{w_1}$, the list $\mathcal{K}_1$ contains ${}^n P_{w_1}$ elements, and forming each element requires dw_1 operations, giving a contribution

$$\mathcal{O}(dw_1 |\mathcal{K}_1|).$$

Likewise, $|\mathcal{S}_{w_2}(\mathbf{b})| = {}^n P_{w_2}$ and $|\mathcal{K}_2| = {}^n P_{w_2}$, producing an additional contribution

$$\mathcal{O}(dw_2 |\mathcal{K}_2|).$$

As Eq. (6) consists of d independent equations, the expected number of collisions between $\mathcal{K}_1$ and $\mathcal{K}_2$ is

$$\frac{|\mathcal{K}_1| |\mathcal{K}_2|}{q^d}.$$

Each collision must be checked for compatibility to form $\mathcal{K}$, and this requires the cost

$$\mathcal{O}\left(\min\{w_1, w_2\} \frac{|\mathcal{K}_1| |\mathcal{K}_2|}{q^d}\right).$$

Consequently, the total time needed to determine all valid solutions of Eq. (6), that is, to generate the set $\mathcal{K}$, is

$$T_{\mathcal{K}} = \mathcal{O}\left(dw_1|\mathcal{K}_1| + dw_2|\mathcal{K}_2| + \min\{w_1, w_2\} \frac{|\mathcal{K}_1||\mathcal{K}_2|}{q^d}\right).$$

After filtration, Eq. (6) yields an expected $\frac{{}^nP_w}{q^d}$ compatible vectors, so that $|\mathcal{K}| = |\mathcal{L}_2| = \frac{{}^nP_w}{q^d}$. Since the last d equations of Eq. (11) are linearly dependent with Eq. (6), only the first $k-d$ equations need to be evaluated. For each $\tilde{\mathbf{x}} \in \mathcal{L}_2$, evaluating these equations requires $(k-d)w$ operations, and therefore

$$\mathcal{O}((k-d)w|\mathcal{L}_2|)$$

is required for generating $\mathcal{L}_2$. Next, since $|\mathcal{S}_{n-w-m+k}(\mathbf{b})| = {}^nP_{n-w-m+k}$, the list $\mathcal{L}_1$ has size ${}^nP_{n-w-m+k}$, and forming each element requires $(k-d)(n-w-m+k)$ operations. This yields the contribution

$$\mathcal{O}((k-d)(n-w-m+k)|\mathcal{L}_1|).$$

Eq. (11) contains $k-d$ independent equations, so the expected number of collisions between $\mathcal{L}_1$ and $\mathcal{L}_2$ is

$$\frac{|\mathcal{L}_1||\mathcal{L}_2|}{q^{k-d}}.$$

Each collision must be checked for compatibility to form the list $\mathcal{L}$, requiring the time

$$\mathcal{O}\left(\min\{n-w-m+k, w\} \cdot \frac{|\mathcal{L}_1||\mathcal{L}_2|}{q^{k-d}}\right).$$

Thus, the total cost of forming $\mathcal{L}$ is

$$T_{\mathcal{L}} = \mathcal{O}\left((k-d)(n-w-m+k)|\mathcal{L}_1| + (k-d)w|\mathcal{L}_2| + \min\{n-w-m+k, w\} \cdot \frac{|\mathcal{L}_1||\mathcal{L}_2|}{q^{k-d}}\right).$$

After filtration, the expected size of $\mathcal{L}$ is $\frac{{}^nP_{n-m+k}}{q^k}$. For each element $(\hat{\mathbf{x}}_2 \parallel \tilde{\mathbf{x}}) \in \mathcal{L}$, the vector $\hat{\mathbf{x}}_1$ is computed using Eq. (10), and the compatibility of the full vector is verified. These operations contribute

$$\mathcal{O}(((m-k)(n-m+k) + \min\{m-k, n-m+k\}) \cdot |\mathcal{L}|).$$

Combining all contributions, the SBC algorithm runs in time

$$T_{\text{SBC}} = \mathcal{O}\left(T_{\text{ISD}}^{(d)}(n, m, w) + T_{\mathcal{K}} + T_{\mathcal{L}} + ((m-k)(n-m+k) + \min\{m-k, n-m+k\}) \cdot |\mathcal{L}|\right).$$

We now discuss the memory complexity of the SBC algorithm. Only the list $\mathcal{K}_1$ and whichever of $\mathcal{L}_1$ or $\mathcal{L}_2$ is smaller are stored explicitly, the remaining lists are generated on the fly and therefore do not contribute to the required memory usage.

Each vector $\tilde{\mathbf{x}}_1$ in $\mathcal{K}_1$ has length w_1 , and storing its ordered I-Supp set requires $w_1 \log_2(n)$ bits. Since $\tilde{\mathbf{H}}_1 \tilde{\mathbf{x}}_1$ is used only for hashing, it does not need to be stored. Hence the memory required for storing $\mathcal{K}_1$ is

$$\mathcal{O}(w_1 \log_2(n) \cdot |\mathcal{K}_1|).$$

Similarly, storing the vectors in $\mathcal{L}_1$ and $\mathcal{L}_2$ requires

$$\mathcal{O}\left((n - w - m + k) \log_2(n) \cdot |\mathcal{L}_1|\right) \quad \text{and} \quad \mathcal{O}\left(w \log_2(n) \cdot |\mathcal{L}_2|\right),$$

respectively. Since only the smaller of $\mathcal{L}_1$ and $\mathcal{L}_2$ is kept in memory, the overall memory requirement of the SBC algorithm becomes

$$M_{\text{SBC}} = \mathcal{O}\left(w_1 \log_2(n) \cdot |\mathcal{K}_1| + \min\{(n - w - m + k) \log_2(n) \cdot |\mathcal{L}_1|, w \log_2(n) \cdot |\mathcal{L}_2|\}\right).$$

H Adaptation of Our Strategy to the SBC Algorithm

We now adapt the ideas of Section 4 to the SBC algorithm. As in our earlier method, the key strategies are the introduction of Compatibility Tables and a constrained enumeration procedure for the vectors $\tilde{\mathbf{x}}$ and $\hat{\mathbf{x}}_2$, which together enable an efficient solution retrieval of Eq. (6) and Eq. (11).

For optimized parameters μ_1 and μ_2 , we begin by selecting at random two disjoint subsets

$$A_1, A_2 \subseteq \text{Supp}(\mathbf{b}), \quad |A_1| = \mu_1, \quad |A_2| = \mu_2,$$

and set $\Lambda = A_1 \cup A_2$. These subsets guide the enumeration of admissible vectors. We consider $\tilde{\mathbf{x}} \in \mathcal{S}_w(\mathbf{b})$ satisfying

$$A_1 \subseteq \text{Supp}(\tilde{\mathbf{x}}) \quad \text{and} \quad \text{Supp}(\tilde{\mathbf{x}}) \setminus A_1 \subseteq \text{Supp}(\mathbf{b}) \setminus \Lambda,$$

and vectors $\hat{\mathbf{x}}_2 \in \mathcal{S}_{n-m+k-w}(\mathbf{b})$ satisfying

$$A_2 \subseteq \text{Supp}(\hat{\mathbf{x}}_2) \quad \text{and} \quad \text{Supp}(\hat{\mathbf{x}}_2) \setminus A_2 \subseteq \text{Supp}(\mathbf{b}) \setminus \Lambda.$$

As in our prior approach, the selection of A_1 and A_2 together with the corresponding constrained enumeration is repeated until a valid solution is obtained. Before describing a single iteration of this enumeration procedure, we first explain the construction of the Compatibility Tables.

Since the SBC algorithm employs a two-stage meet-in-the-middle approach, each stage requires its own compatibility filtration and we therefore construct one Compatibility Table per stage. For the first meet-in-the-middle stage, we define a table $\mathcal{A}^{(1)} \in \{0, 1\}^{\binom{n}{w_1} \times \binom{n}{w_2}}$ as follows: For each subset $S_1 \subseteq [n]$ of size w_1 , we compute its index $i = \rho_{w_1}(S_1)$, for every subset $S_2 \subseteq [n] \setminus S_1$ of size w_2 , we compute $j = \rho_{w_2}(S_2)$ and set $\mathcal{A}^{(1)}[i][j] = 1$, while all remaining entries remain equal to 0. Analogously, for the second meet-in-the-middle stage, we

define $\mathcal{A}^{(2)} \in \{0, 1\}^{\binom{n}{n-w-m+k} \times \binom{n}{w}}$ using the same rule adapted to the support sizes relevant for Eq. (11).

We now describe a single iteration of the enumeration process. If $\Lambda_1 \subseteq \text{Supp}(\tilde{\mathbf{x}}^*)$, the support of $\tilde{\mathbf{x}}^*$ must distribute the elements of Λ_1 between the supports of $\tilde{\mathbf{x}}_1^*$ and $\tilde{\mathbf{x}}_2^*$. Thus there exists a subset $\Lambda_{11} \subseteq \Lambda_1$ such that

$$\Lambda_{11} \subseteq \text{Supp}(\tilde{\mathbf{x}}_1^*) \quad \text{and} \quad \Lambda_{12} := \Lambda_1 \setminus \Lambda_{11} \subseteq \text{Supp}(\tilde{\mathbf{x}}_2^*).$$

Hence, similar as in Algorithm 2, we iterate over all subsets $\Lambda_{11} \subseteq \Lambda_1$ satisfying

$$|\Lambda_{11}| \leq w_1 \quad \text{and} \quad \mu_1 - |\Lambda_{11}| \leq w_2.$$

For each such choice of Λ_{11} , we generate the following two lists using incremental updates based on the SJT algorithm:

$$\mathcal{K}_{\Lambda_{11}}^{(1)} = \left\{ \left(\tilde{\mathbf{H}}_1 \tilde{\mathbf{x}}_1, \tilde{\mathbf{x}}_1, \rho_{w_1}(\text{I-Supp}(\tilde{\mathbf{x}}_1)) \right) \mid \tilde{\mathbf{x}}_1 \in \mathcal{S}_{w_1}(\mathbf{b}), \Lambda_{11} \subseteq \text{Supp}(\tilde{\mathbf{x}}_1), \text{Supp}(\tilde{\mathbf{x}}_1) \setminus \Lambda_{11} \subseteq \text{Supp}(\mathbf{b}) \setminus \Lambda \right\},$$

$$\mathcal{K}_{\Lambda_{12}}^{(1)} = \left\{ \left(\tilde{\mathbf{t}} - \tilde{\mathbf{H}}_2 \tilde{\mathbf{x}}_2, \tilde{\mathbf{x}}_2, \rho_{w_2}(\text{I-Supp}(\tilde{\mathbf{x}}_2)) \right) \mid \tilde{\mathbf{x}}_2 \in \mathcal{S}_{w_2}(\mathbf{b}), \Lambda_{12} \subseteq \text{Supp}(\tilde{\mathbf{x}}_2), \text{Supp}(\tilde{\mathbf{x}}_2) \setminus \Lambda_{12} \subseteq \text{Supp}(\mathbf{b}) \setminus \Lambda \right\}.$$

To solve Eq. (6), we apply a meet-in-the-middle procedure to the sets $\mathcal{K}_{\Lambda_{11}}^{(1)}$ and $\mathcal{K}_{\Lambda_{12}}^{(1)}$ followed by a compatibility check using the table $\mathcal{A}^{(1)}$. This yields the set

$$\mathcal{K}_{\Lambda_{11}} = \left\{ (\tilde{\mathbf{x}}_1 \parallel \tilde{\mathbf{x}}_2) \in \mathcal{S}_w(\mathbf{b}) \mid \tilde{\mathbf{H}}(\tilde{\mathbf{x}}_1 \parallel \tilde{\mathbf{x}}_2) = \tilde{\mathbf{t}}, \Lambda_{11} \subseteq \text{Supp}(\tilde{\mathbf{x}}_1), \Lambda_{12} \subseteq \text{Supp}(\tilde{\mathbf{x}}_2) \right\}.$$

Taking the union over all admissible Λ_{11} gives

$$\mathcal{K} = \bigcup_{\substack{\Lambda_{11} \subseteq \Lambda_1 \\ |\Lambda_{11}| \leq w_1 \\ \mu_1 - |\Lambda_{11}| \leq w_2}} \mathcal{K}_{\Lambda_{11}}.$$

By construction, the union over all admissible Λ_{11} yields precisely the constrained solutions of Eq. (6), namely

$$\mathcal{K} = \left\{ \tilde{\mathbf{x}} \in \mathcal{S}_w(\mathbf{b}) \mid \tilde{\mathbf{H}}\tilde{\mathbf{x}} = \tilde{\mathbf{t}}, \Lambda_1 \subseteq \text{Supp}(\tilde{\mathbf{x}}), \text{Supp}(\tilde{\mathbf{x}}) \setminus \Lambda_1 \subseteq \text{Supp}(\mathbf{b}) \setminus \Lambda \right\}.$$

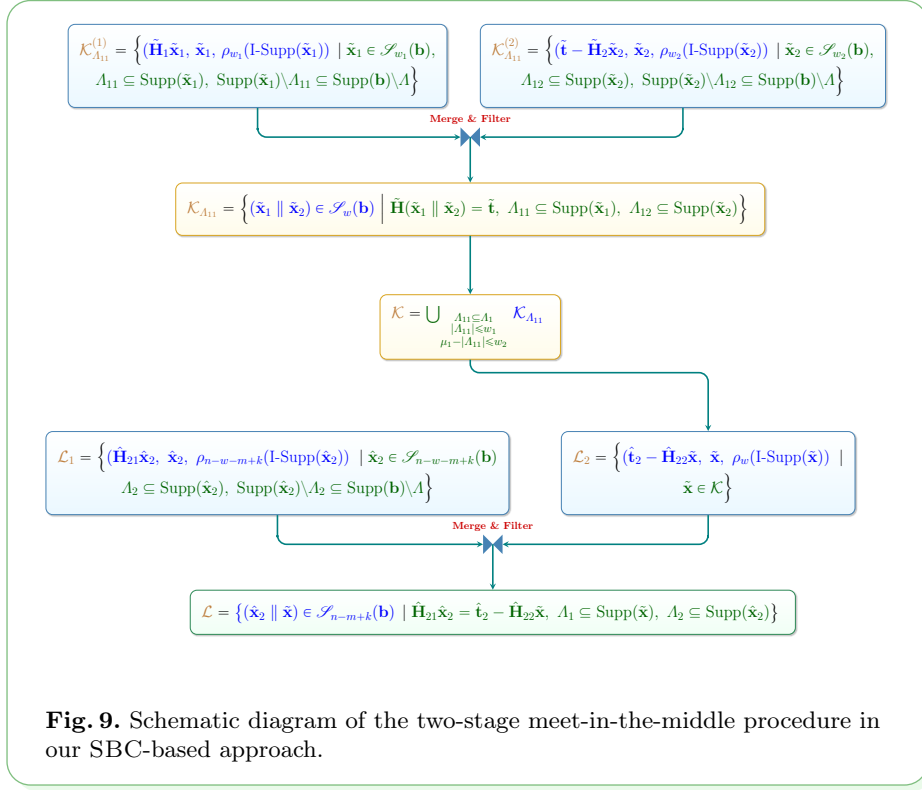
Next, to solve Eq. (11), we use the constraint on $\hat{\mathbf{x}}_2$ and the previously obtained set $\mathcal{K}$ to construct

$$\mathcal{L}_1 = \left\{ \left(\hat{\mathbf{H}}_{21} \hat{\mathbf{x}}_2, \hat{\mathbf{x}}_2, \rho_{n-w-m+k}(\text{I-Supp}(\hat{\mathbf{x}}_2)) \right) \mid \hat{\mathbf{x}}_2 \in \mathcal{S}_{n-w-m+k}(\mathbf{b}), \Lambda_2 \subseteq \text{Supp}(\hat{\mathbf{x}}_2), \right. \\ \left. \text{Supp}(\hat{\mathbf{x}}_2) \setminus \Lambda_2 \subseteq \text{Supp}(\mathbf{b}) \setminus \Lambda \right\},$$

$$\mathcal{L}_2 = \left\{ \left(\hat{\mathbf{t}}_2 - \hat{\mathbf{H}}_{22} \tilde{\mathbf{x}}, \tilde{\mathbf{x}}, \rho_w(\text{I-Supp}(\tilde{\mathbf{x}})) \right) \mid \tilde{\mathbf{x}} \in \mathcal{K} \right\}.$$

Applying the meet-in-the-middle approach on $\mathcal{L}_1$ and $\mathcal{L}_2$ followed by a filtration using the Compatibility Table $\mathcal{A}^{(2)}$, we obtain the set

$$\mathcal{L} = \left\{ (\hat{\mathbf{x}}_2 \parallel \tilde{\mathbf{x}}) \in \mathcal{S}_{n-m+k}(\mathbf{b}) \mid \hat{\mathbf{H}}_{21} \hat{\mathbf{x}}_2 = \hat{\mathbf{t}}_2 - \hat{\mathbf{H}}_{22} \tilde{\mathbf{x}}, \Lambda_1 \subseteq \text{Supp}(\tilde{\mathbf{x}}), \Lambda_2 \subseteq \text{Supp}(\hat{\mathbf{x}}_2) \right\}.$$



This two-stage meet-in-the-middle procedure is depicted in Figure 9.

Finally, for each vector in $\mathcal{L}$, the full vector $\mathbf{Sx}$ is constructed and if $\mathbf{Sx} \in \mathcal{S}_n(\mathbf{b})$ holds, the corresponding vector $\mathbf{x}$ is a valid solution for the PKP instance. If the current iteration yields no solution, the algorithm resamples (A_1, A_2) and repeats the procedure until a valid solution is produced. The pseudocode of the overall procedure is presented in Algorithm 3.

Complexity Analysis. First, we analyze the time and memory complexity of the precomputation phase. In this phase, we generate the swap_sequences corresponding to the integers w_1, w_2 , and $n - w - m + k$. This step requires time

$$\mathcal{O}(w_1 \cdot w_1! + w_2 \cdot w_2! + (n - w - m + k) \cdot (n - w - m + k)!),$$

and memory (in bits)

$$\mathcal{O}(\log_2(w_1) \cdot w_1! + \log_2(w_2) \cdot w_2! + \log_2(n - w - m + k) \cdot (n - w - m + k)!).$$

Furthermore, as in the previous analysis, the cost of constructing the Compatibility Table $\mathcal{A}^{(1)} \in \{0, 1\}^{\binom{n}{w_1} \times \binom{n}{w_2}}$ is

$$\mathcal{O}\left(w_2 \log_2\left(\binom{n}{w_2}\right) \cdot \binom{n}{w_1} \binom{n - w_1}{w_2}\right).$$

Algorithm 3: Our approach with *SBC* Algorithm

Input: $(\tilde{\mathbf{A}}, \mathbf{b}) \in \mathbb{F}_q^{m \times n} \times \mathbb{F}_q^n$

- 1 Choose optimal $d, w, w_1, k, \mu_1, \mu_2$, such that
 $1 \leq d \leq m, 1 \leq w \leq n, 1 \leq w_1 \leq w, d \leq k \leq m, 0 \leq \mu_1 \leq w, 0 \leq \mu_2 \leq n - m + k - w$, and
 compute $w_2 = w - w_1$;
- 2 Use ISD to find a system $(\tilde{\mathbf{H}}, \tilde{\mathbf{t}})$ with d equations and w variables;
- 3 Set $\tilde{\mathbf{H}}_1 \leftarrow \tilde{\mathbf{H}}[:, w_1]$, and $\tilde{\mathbf{H}}_2 \leftarrow \tilde{\mathbf{H}}[w_1 :]$;
- 4 Find a permutation matrix $\mathbf{S}$ such that the variables involved in the system $(\tilde{\mathbf{H}}, \tilde{\mathbf{t}})$ are shifted to the last w co-ordinates in the given system;
- 5 Find an invertible matrix $\tilde{\mathbf{Q}}$ such that $\tilde{\mathbf{Q}}\tilde{\mathbf{A}}\mathbf{S}^{-1} = \begin{pmatrix} \mathbf{I}_{m-k} & \tilde{\mathbf{H}}_1 \\ \mathbf{0} & \tilde{\mathbf{H}}_2 \end{pmatrix}$, and compute $\hat{\mathbf{t}} = \tilde{\mathbf{Q}}(\sigma, 0, \dots, 0)^T$,
 $\sigma = \sum_i b_i$;
- 6 Set $\tilde{\mathbf{H}}_{21} \leftarrow \tilde{\mathbf{H}}_2[:, n - w - m + k]$, and $\tilde{\mathbf{H}}_{22} \leftarrow \tilde{\mathbf{H}}_2[n - w - m + k :]$;
- 7 Set $\tilde{\mathbf{t}}_1 \leftarrow \tilde{\mathbf{t}}[:, m - k]$, and $\tilde{\mathbf{t}}_2 \leftarrow \tilde{\mathbf{t}}[m - k :]$;
- 8 Set $\mathcal{A}^{(1)} \leftarrow \mathbf{0} \in \{0, 1\}^{\binom{n}{w_1} \times \binom{n}{w_2}}$ // Compatibility Table for Stage-1 meet-in-the-middle
- 9 **for** $S_1 \subseteq [n]$ of size w_1 **do**
- 10 $i \leftarrow \rho_{w_1}(S_1)$;
- 11 **for** $S_2 \subseteq [n] \setminus S_1$ of size w_2 **do**
- 12 $j \leftarrow \rho_{w_2}(S_2)$;
- 13 $\mathcal{A}^{(1)}[i][j] \leftarrow 1$;
- 14 Set $\mathcal{A}^{(2)} \leftarrow \mathbf{0} \in \{0, 1\}^{\binom{n}{n-w-m+k} \times \binom{n}{w}}$ // Compatibility Table for Stage-2 meet-in-the-middle
- 15 **for** $S_1 \subseteq [n]$ of size $n - w - m + k$ **do**
- 16 $i \leftarrow \rho_{n-w-m+k}(S_1)$;
- 17 **for** $S_2 \subseteq [n] \setminus S_1$ of size w **do**
- 18 $j \leftarrow \rho_w(S_2)$;
- 19 $\mathcal{A}^{(2)}[i][j] \leftarrow 1$;
- 20 **repeat**
- 21 Choose disjoint random subsets $A_1, A_2 \subseteq \text{Supp}(\mathbf{b})$ such that $|A_1| = \mu_1$ and $|A_2| = \mu_2$;
- 22 Compute $A \leftarrow A_1 \cup A_2$;
- 23 $\mathcal{K} \leftarrow \emptyset$;
- 24 **for all** $A_{11} \subseteq A_1$ such that $|A_{11}| \leq w_1$ and $\mu_1 - |A_{11}| \leq w_2$ **do**
- 25 $A_{12} \leftarrow A_1 \setminus A_{11}$;
- 26 // Lists $\mathcal{K}_{A_{11}}^{(1)}$ and $\mathcal{K}_{A_{12}}^{(1)}$ are generated via the SJT algorithm with incremental updates
 (Section 4.1), using hash maps.
- 27 $\mathcal{K}_{A_{11}}^{(1)} \leftarrow \left\{ \left(\tilde{\mathbf{H}}_1 \tilde{\mathbf{x}}_1, \tilde{\mathbf{x}}_1, \rho_{w_1}(\text{I-Supp}(\tilde{\mathbf{x}}_1)) \right) \mid \tilde{\mathbf{x}}_1 \in \mathcal{S}_{w_1}(\mathbf{b}), A_{11} \subseteq \text{Supp}(\tilde{\mathbf{x}}_1), \text{Supp}(\tilde{\mathbf{x}}_1) \setminus A_{11} \subseteq \text{Supp}(\mathbf{b}) \setminus A \right\}$;
- 28 $\mathcal{K}_{A_{12}}^{(1)} \leftarrow \left\{ \left(\tilde{\mathbf{t}} - \tilde{\mathbf{H}}_2 \tilde{\mathbf{x}}_2, \tilde{\mathbf{x}}_2, \rho_{w_2}(\text{I-Supp}(\tilde{\mathbf{x}}_2)) \right) \mid \tilde{\mathbf{x}}_2 \in \mathcal{S}_{w_2}(\mathbf{b}), A_{12} \subseteq \text{Supp}(\tilde{\mathbf{x}}_2), \text{Supp}(\tilde{\mathbf{x}}_2) \setminus A_{12} \subseteq \text{Supp}(\mathbf{b}) \setminus A \right\}$;
- 29 $\mathcal{K}_{A_{11}} \leftarrow \emptyset$;
- 30 **for** $(\mathbf{c}, \tilde{\mathbf{x}}_1, i) \in \mathcal{K}_{A_{11}}^{(1)}$ and $(\mathbf{d}, \tilde{\mathbf{x}}_2, j) \in \mathcal{K}_{A_{12}}^{(1)}$ **do**
- 31 **if** $\mathbf{c} = \mathbf{d}$ and $\mathcal{A}^{(1)}[i][j] = 1$ **then**
- 32 $\mathcal{K}_{A_{11}} \leftarrow \mathcal{K}_{A_{11}} \cup \{(\tilde{\mathbf{x}}_1 \parallel \tilde{\mathbf{x}}_2)\}$;
- 33 $\mathcal{K} \leftarrow \mathcal{K} \cup \mathcal{K}_{A_{11}}$;
- 34 // The list $\mathcal{L}_1$ is generated via the SJT algorithm with incremental updates (Section 4.1),
 using hash maps.
- 35 $\mathcal{L}_1 \leftarrow \left\{ \left(\tilde{\mathbf{H}}_{21} \tilde{\mathbf{x}}_2, \tilde{\mathbf{x}}_2, \rho_{n-w-m+k}(\text{I-Supp}(\tilde{\mathbf{x}}_2)) \right) \mid \tilde{\mathbf{x}}_2 \in \mathcal{S}_{n-w-m+k}(\mathbf{b}), A_2 \subseteq \text{Supp}(\tilde{\mathbf{x}}_2), \text{Supp}(\tilde{\mathbf{x}}_2) \setminus A_2 \subseteq \text{Supp}(\mathbf{b}) \setminus A \right\}$;
- 36 $\mathcal{L}_2 \leftarrow \left\{ \left(\tilde{\mathbf{t}}_2 - \tilde{\mathbf{H}}_{22} \tilde{\mathbf{x}}, \tilde{\mathbf{x}}, \rho_w(\text{I-Supp}(\tilde{\mathbf{x}})) \right) \mid \tilde{\mathbf{x}} \in \mathcal{K} \right\}$;
- 37 **for** $(\mathbf{c}, \tilde{\mathbf{x}}_2, i) \in \mathcal{L}_1$ and $(\mathbf{d}, \tilde{\mathbf{x}}, j) \in \mathcal{L}_2$ **do**
- 38 **if** $\mathbf{c} = \mathbf{d}$ and $\mathcal{A}^{(2)}[i][j] = 1$ **then**
- 39 $\tilde{\mathbf{y}} \leftarrow (\tilde{\mathbf{x}}_2 \parallel \tilde{\mathbf{x}})$;
- 40 $\tilde{\mathbf{x}}_1 \leftarrow \tilde{\mathbf{t}}_1 - \tilde{\mathbf{H}}_1 \tilde{\mathbf{y}}$;
- 41 **if** $(\tilde{\mathbf{x}}_1 \parallel \tilde{\mathbf{y}}) \in \mathcal{S}_n(\mathbf{b})$ // Check compatibility via intersection
 then
 return $\mathbf{S}^{-1}(\tilde{\mathbf{x}}_1 \parallel \tilde{\mathbf{y}})$;
- 42 **until** Solution found;

Similarly, the cost of constructing the Compatibility Table $\mathcal{A}^{(2)} \in \{0, 1\}^{\binom{n}{n-w-m+k} \times \binom{n}{w}}$ is

$$\mathcal{O}\left(w \log_2\left(\binom{n}{w}\right) \cdot \binom{n}{n-w-m+k} \binom{w+m-k}{w}\right).$$

Combining these contributions, the total time complexity of the precomputation phase is

$$\begin{aligned} T_{\text{precompute}} = & \mathcal{O}\left(w_1 \cdot w_1! + w_2 \cdot w_2! + (n-w-m+k) \cdot (n-w-m+k)!\right. \\ & + w_2 \log_2\left(\binom{n}{w_2}\right) \cdot \binom{n}{w_1} \binom{n-w_1}{w_2} \\ & \left. + w \log_2\left(\binom{n}{w}\right) \cdot \binom{n}{n-w-m+k} \binom{w+m-k}{w}\right). \end{aligned}$$

The corresponding memory complexity (in bits) of the precomputation phase is

$$\begin{aligned} M_{\text{precompute}} = & \mathcal{O}\left(\log_2(w_1) \cdot w_1! + \log_2(w_2) \cdot w_2!\right. \\ & + \log_2(n-w-m+k) \cdot (n-w-m+k)! \\ & \left. + \binom{n}{w_1} \binom{n}{w_2} + \binom{n}{n-w-m+k} \binom{n}{w}\right). \end{aligned}$$

We now examine the remaining complexity of the algorithm. As noted earlier, the cost of identifying d equations supported on w variables using ISD techniques is

$$T_{\text{ISD}}^{(d)}(n, m, w) = \mathcal{O}\left(\frac{m^3 + \binom{m}{d}}{p^{(d)}(n, m, w)}\right) \quad \text{where,}$$

$$p^{(d)}(n, m, w) = \min\left\{\frac{\binom{w}{d} \binom{n-w}{m-d}}{\binom{n}{m}} \check{N}_{w,d}, 1\right\}, \quad \check{N}_{w,d} = \binom{n}{w} (q^d - 1)^{w-d} \prod_{i=0}^{d-1} \frac{1 - q^{m-i}}{1 - q^{i+1}} \quad \text{and,} \quad \check{N}_{w,d} \geq 1.$$

The remaining time complexity of Algorithm 3 is given by the number of repetitions needed for success, multiplied by the cost of a single execution. We therefore begin by analyzing the complexity of one iteration.

Consider Eq. (6) and fix a subset Λ_{11} with $|\Lambda_{11}| = i$. The list $\mathcal{K}_{\Lambda_{11}}^{(1)}$ contains $\binom{n-\mu_1-\mu_2}{w_1-i} w_1!$ elements, corresponding to $\binom{n-\mu_1-\mu_2}{w_1-i}$ distinct supports. Using the incremental update technique together with the ρ_{w_1} function, the cost of generating this list becomes

$$T_{\mathcal{K}_{\Lambda_{11}}^{(1)}} = \mathcal{O}\left((w_1 \log_2\left(\binom{n}{w_1}\right) + d \cdot w_1!) \binom{n-\mu_1-\mu_2}{w_1-i}\right).$$

Similarly, the list $\mathcal{K}_{\Lambda_{12}}^{(1)}$ has cardinality

$$|\mathcal{K}_{\Lambda_{12}}^{(1)}| = \binom{n-\mu_1-\mu_2}{w_2-(\mu_1-i)} w_2!,$$

which contributes an additional cost

$$T_{\mathcal{K}_{\Lambda_{12}}^{(1)}} = \mathcal{O}\left((w_2 \log_2 \binom{n}{w_2}) + d \cdot w_2!\right) \binom{n - \mu_1 - \mu_2}{w_2 - (\mu_1 - i)}.$$

As Eq. (6) consists of d independent equations, the expected number of collisions between $\mathcal{K}_{\Lambda_{11}}^{(1)}$ and $\mathcal{K}_{\Lambda_{12}}^{(1)}$ is

$$\frac{|\mathcal{K}_{\Lambda_{11}}^{(1)}| |\mathcal{K}_{\Lambda_{12}}^{(1)}|}{q^d}.$$

To obtain the final list $\mathcal{K}_{\Lambda_{11}}$, we test each collision for compatibility using the precomputed table $\mathcal{A}^{(1)}$. Consequently, the total time required to construct the list $\mathcal{K}_{\Lambda_{11}}$ is

$$T_i = \mathcal{O}\left(T_{\mathcal{K}_{\Lambda_{11}}^{(1)}} + T_{\mathcal{K}_{\Lambda_{12}}^{(1)}} + \frac{|\mathcal{K}_{\Lambda_{11}}^{(1)}| |\mathcal{K}_{\Lambda_{12}}^{(1)}|}{q^d}\right).$$

Now, $|\Lambda_1| = \mu_1$ and we iterate over all subsets $\Lambda_{11} \subseteq \Lambda_1$ with $|\Lambda_{11}| = i$, satisfying

$$|\Lambda_{11}| \leq w_1 \quad \text{and} \quad \mu_1 - |\Lambda_{11}| \leq w_2.$$

Consequently, the total time needed to determine all solutions of Eq. (6), and thus to generate the list $\mathcal{K}$, is

$$T_{\mathcal{K}} = \mathcal{O}\left(\sum_{\max\{0, \mu_1 - w_2\} \leq i \leq \min\{\mu_1, w_1\}} \binom{\mu_1}{i} T_i\right).$$

Since, $\mathcal{K}$ contains all the solutions $\tilde{\mathbf{x}}$ of Eq. (6) such that $\Lambda_1 \subseteq \text{Supp}(\tilde{\mathbf{x}})$ and $\text{Supp}(\tilde{\mathbf{x}}) \setminus \Lambda_1 \subseteq \text{Supp}(\mathbf{b}) \setminus \Lambda$, the expected size of the list is

$$|\mathcal{K}| = \frac{\binom{n - \mu_1 - \mu_2}{w - \mu_1} w!}{q^d}.$$

To construct $\mathcal{L}_1$, note that the support of any valid vector $\hat{\mathbf{x}}_2$ must include the fixed subset Λ_2 . The remaining $n - w - m + k - \mu_2$ positions must be selected from $\text{Supp}(\mathbf{b}) \setminus \Lambda$, which contains $n - \mu_1 - \mu_2$ elements. Hence, the number of admissible supports for $\hat{\mathbf{x}}_2$ is $\binom{n - \mu_1 - \mu_2}{n - w - m + k - \mu_2}$. Each such support gives rise to $(n - w - m + k)!$ distinct vectors $\hat{\mathbf{x}}_2$, implying that

$$|\mathcal{L}_1| = \binom{n - \mu_1 - \mu_2}{n - w - m + k - \mu_2} (n - w - m + k)!.$$

The cost of constructing the list $\mathcal{L}_1$ using incremental updates together with the rank function is

$$T_{\mathcal{L}_1} = \mathcal{O}\left((n - w - m + k) \log_2 \binom{n}{n - w - m + k} + (k - d) \cdot (n - w - m + k)! \binom{n - \mu_1 - \mu_2}{n - w - m + k - \mu_2}\right).$$

The list $\mathcal{L}_2$ is derived directly from $\mathcal{K}$. Consequently, its cardinality satisfies

$$|\mathcal{L}_2| = |\mathcal{K}| = \frac{\binom{n - \mu_1 - \mu_2}{w - \mu_1} w!}{q^d}.$$

Generating a single element of $\mathcal{L}_2$ requires $(k-d)w$ operations. Hence, the total cost of constructing $\mathcal{L}_2$ is

$$T_{\mathcal{L}_2} = \mathcal{O}((k-d)w \cdot |\mathcal{L}_2|).$$

Since Eq. (11) consists of $k-d$ independent equations, the expected number of collisions between $\mathcal{L}_1$ and $\mathcal{L}_2$ is

$$\frac{|\mathcal{L}_1| |\mathcal{L}_2|}{q^{k-d}}.$$

To construct the list $\mathcal{L}$, each collision must be checked for compatibility using compatibility matrix $\mathcal{A}^{(2)}$, requiring the overall cost

$$T_{\mathcal{L}} = \mathcal{O}\left(T_{\mathcal{L}_1} + T_{\mathcal{L}_2} + \frac{|\mathcal{L}_1| |\mathcal{L}_2|}{q^{k-d}}\right).$$

Due to the constraints mentioned earlier, the expected size of $\mathcal{L}$ becomes

$$|\mathcal{L}| = \frac{\binom{n-\mu_1-\mu_2}{w-\mu_1} w! \binom{n-w-\mu_2}{n-w-m+k-\mu_2} (n-w-m+k)!}{q^k}.$$

For each element of $\mathcal{L}$, we compute the corresponding vector $\hat{\mathbf{x}}_1$ and verify its compatibility in order to recover the solution. This requires a cost of $(n-m+k)(m-k) + \min\{m-k, n-m+k\}$. Hence, the total time needed to retrieve the solution from the list $\mathcal{L}$ becomes

$$((n-m+k)(m-k) + \min\{m-k, n-m+k\}) |\mathcal{L}|.$$

Therefore, the total time complexity of a single iteration becomes

$$T_{\text{iter}} = \mathcal{O}\left(T_{\mathcal{K}} + T_{\mathcal{L}} + ((n-m+k)(m-k) + \min\{m-k, n-m+k\}) |\mathcal{L}|\right).$$

To determine the number of repetitions needed for success, observe that a random pair of disjoint subsets (A_1, A_2) of $\text{Supp}(\mathbf{b})$ satisfies $A_1 \subseteq \text{Supp}(\hat{\mathbf{x}}^*)$ and $A_2 \subseteq \text{Supp}(\hat{\mathbf{x}}_2^*)$ with probability

$$p = \frac{\binom{w}{\mu_1} \binom{n-w-m+k}{\mu_2}}{\binom{n}{\mu_1} \binom{n-\mu_1}{\mu_2}}.$$

Therefore, the expected number of repetitions is $1/p$, and the overall cost becomes

$$T_{\text{Alg-3}} = \mathcal{O}\left(T_{\text{precompute}} + T_{\text{ISD}}^{(d)}(n, m, w) + \frac{1}{p} \cdot T_{\text{iter}}\right).$$

Next, we consider the memory required to store the list $\mathcal{K}_{A_{11}}^{(1)}$ of maximum size among all $A_{11} \subseteq A_1$ satisfying $\max\{0, \mu_1 - w_2\} \leq |A_{11}| \leq \min\{\mu_1, w_1\}$. Additionally, between $\mathcal{L}_1$ and $\mathcal{L}_2$, only the smaller list is kept in memory, the

other lists are generated on the fly and thus do not contribute to the total memory complexity.

Each vector $\tilde{\mathbf{x}}_1 \in \mathcal{K}_{\Lambda_{11}}^{(1)}$ has weight w_1 , and storing its ordered I-Supp requires $w_1 \log_2(n)$ bits. Since the value $\tilde{\mathbf{H}}_1 \tilde{\mathbf{x}}_1$ is used solely for hashing purposes, it does not need to be stored. Hence the memory required for storing all such $\mathcal{K}_{\Lambda_{11}}^{(1)}$ is

$$\mathcal{O}\left(\max_{\Lambda_{11}} \left\{ |\mathcal{K}_{\Lambda_{11}}^{(1)}| \cdot (w_1 \log_2(n) + \log_2 \left(\binom{n}{w_1} \right)) \right\}\right).$$

Similarly, storing the vectors in $\mathcal{L}_1$ and $\mathcal{L}_2$ requires

$$\mathcal{O}\left(\left((n - w - m + k) \log_2(n) + \log_2 \left(\binom{n}{n - w - m + k} \right)\right) \cdot |\mathcal{L}_1|\right) \quad \text{and} \quad \mathcal{O}\left((w \log_2 n + \log_2 \left(\binom{n}{w} \right)) \cdot |\mathcal{L}_2|\right),$$

respectively. Since only the smaller of $\mathcal{L}_1$ and $\mathcal{L}_2$ is kept in memory, the overall memory requirement (in bits) of the Algorithm 3 becomes

$$\begin{aligned} M_{\text{Alg-3}} = & \mathcal{O}\left(M_{\text{precomputed}} + \max_{\Lambda_{11}} \left\{ |\mathcal{K}_{\Lambda_{11}}^{(1)}| \cdot \left(w_1 \log_2(n) + \log_2 \left(\binom{n}{w_1} \right) \right) \right\} \right. \\ & \left. + \min \left\{ \left((n - w - m + k) \log_2(n) + \log_2 \left(\binom{n}{n - w - m + k} \right) \right) \cdot |\mathcal{L}_1|, \left(w \log_2 n + \log_2 \left(\binom{n}{w} \right) \right) \cdot |\mathcal{L}_2| \right\} \right). \end{aligned}$$